

# Recurrent Neural Networks and LSTMs: Learning from Sequential Data

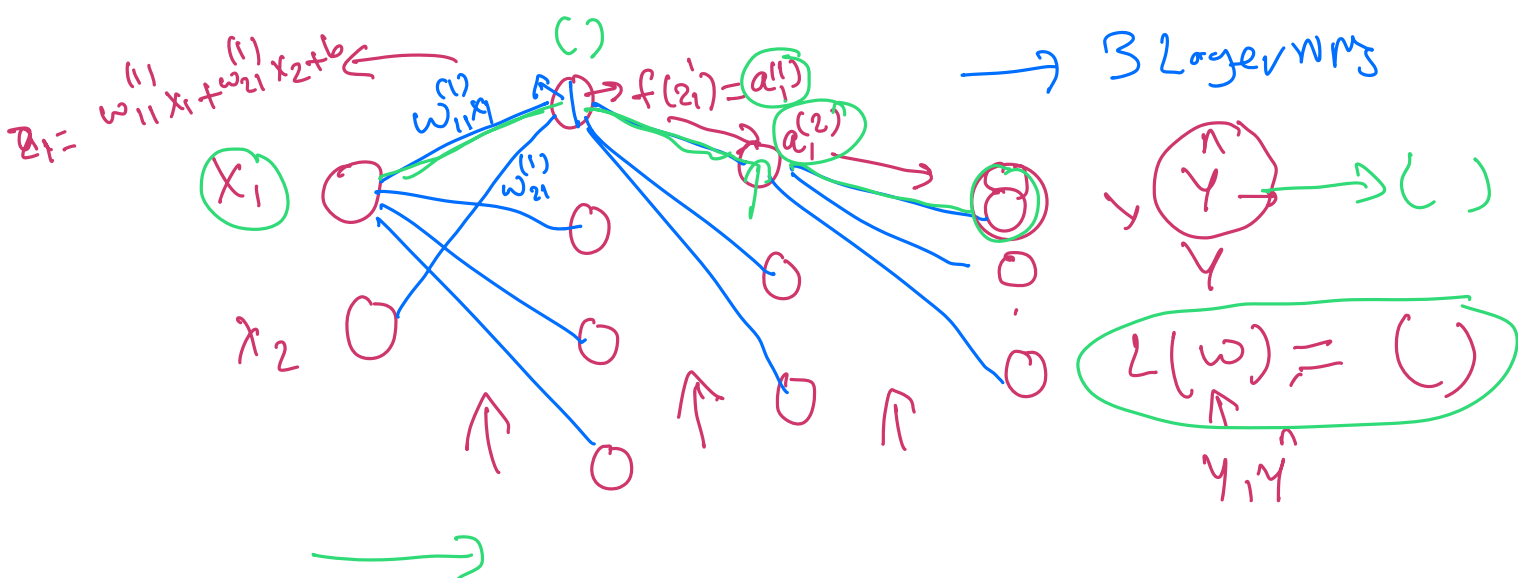
Modeling sequences, memory, and temporal dependencies in  
neural networks

---

by Dr. Rishikesh Yadav

June 25, 2025

Assistant Professor, School of Mathematical and Statistical Sciences, IIT Mandi, India



FP

Feed forward NNS

Ar. NNS

NNS

shallow NNS

BP

GD:  $\rightarrow 1000$

$w^{(k+1)} = w^{(k)} - \eta \nabla L(w^t)$

$n=10$

SGD: ①

mini: 10

$n=1000$

$\downarrow$

example

$\hookrightarrow a_1^{(1)} = \text{relu}(w_{11}^{(1)} x_1 + w_{21}^{(1)} x_2 + \cancel{b})$

$= 0$

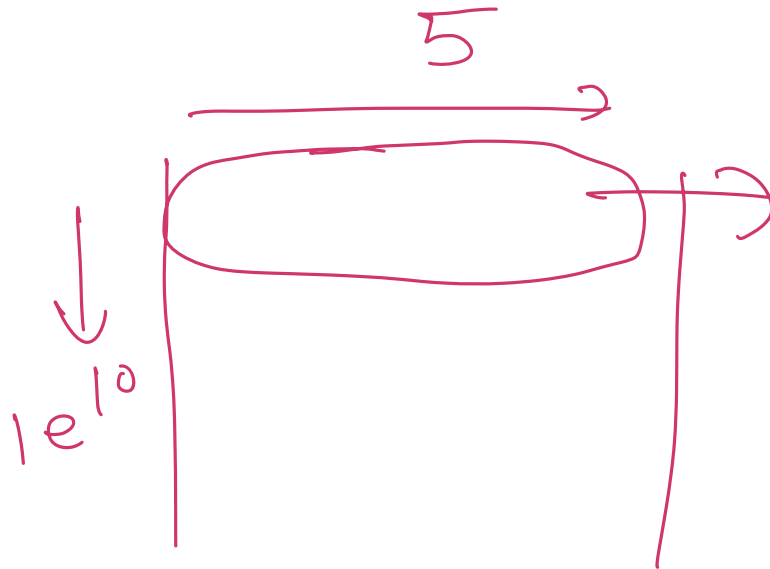
$b^{(t+1)} = b^{(t)} - \eta \nabla L(w^t)$

$w^{(t+1)} \rightarrow b^t \rightarrow$

$w^{(t+1)} = w^{(t)} - \eta \nabla L(w^t)$

$$n = 1e^{10}$$

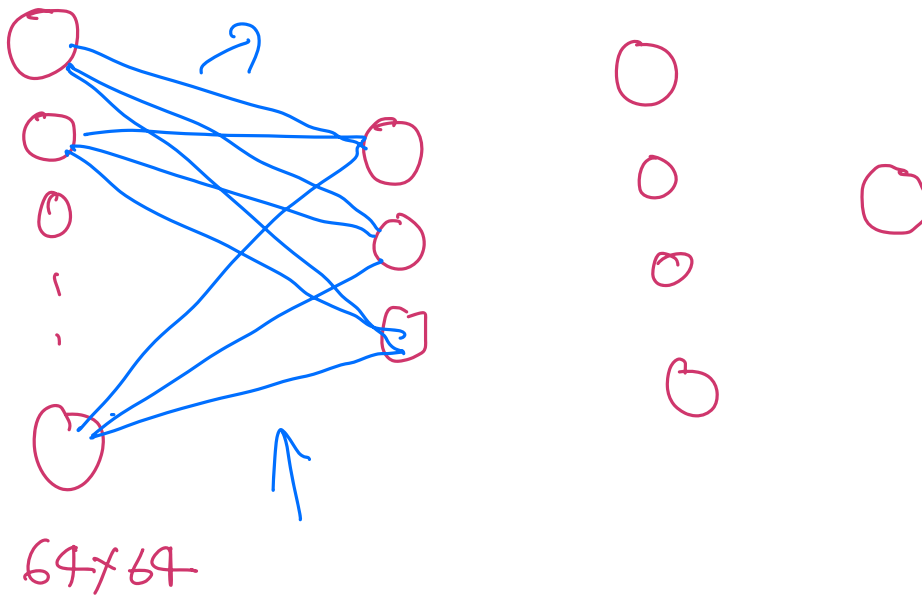
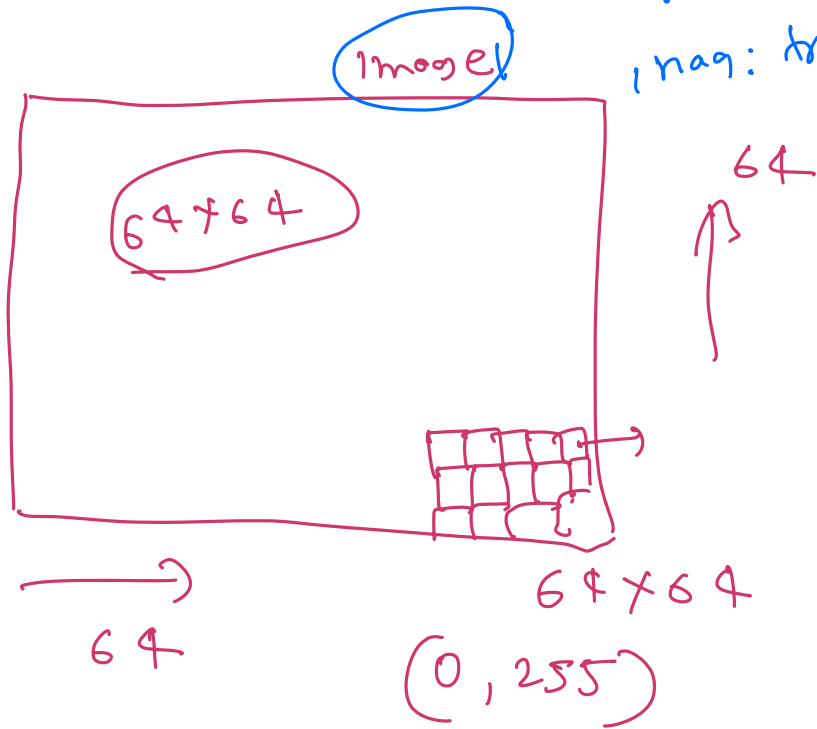
$$D = 5$$



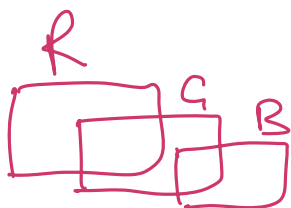
100 Layer nn

$$\overline{f(x)} = \underbrace{f_1 \cdot f_2 \cdot f_3 \dots f_{100}(x)}_{\uparrow} \rightarrow$$

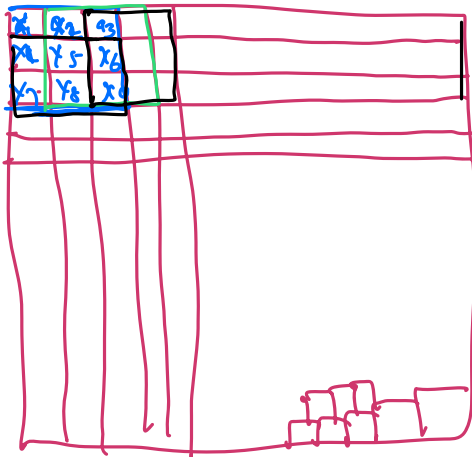
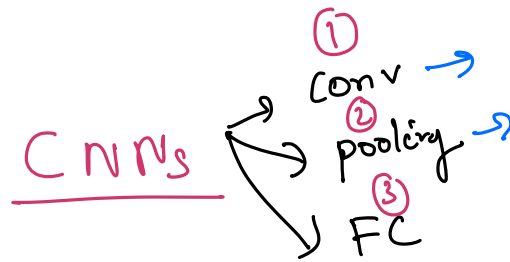
→ CNN (S) → tran. 1 → n=100  
 , n=9: tra 2



$(64 \times 64 \times 3) \times 3$



$\in (0, 255)$



64x64

gray

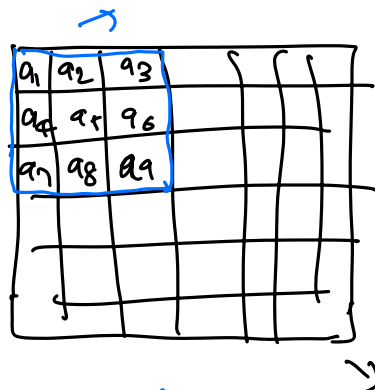
kernels

|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |

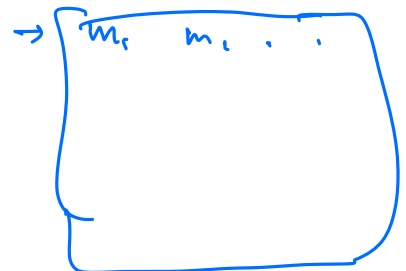
$$\vec{a}_i = \sigma(x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_9 w_9)$$

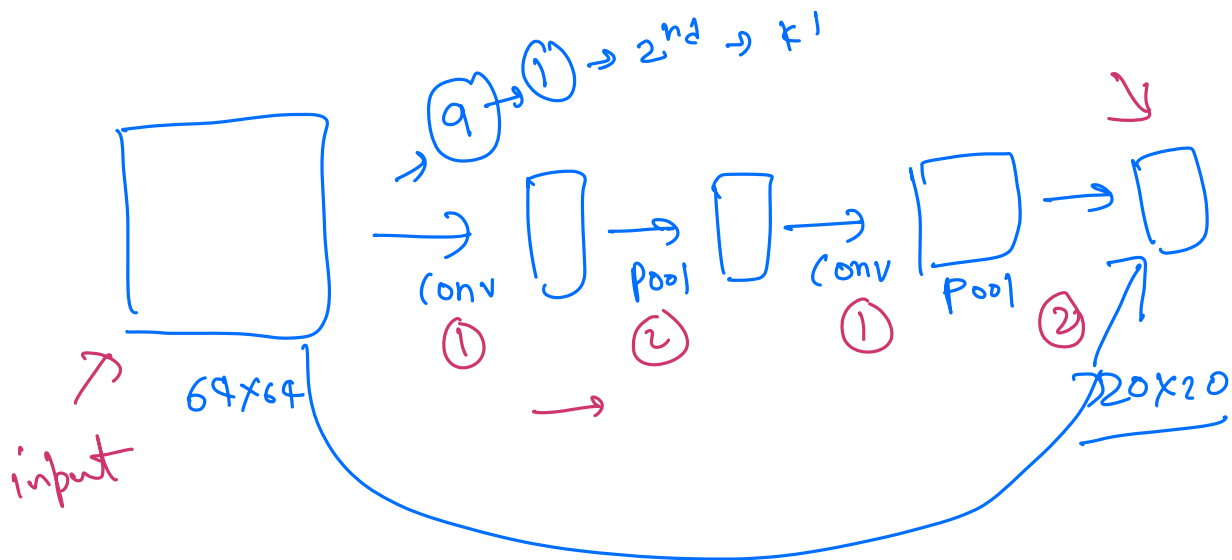
$a_2 =$  \_\_\_\_\_

$a_3 =$

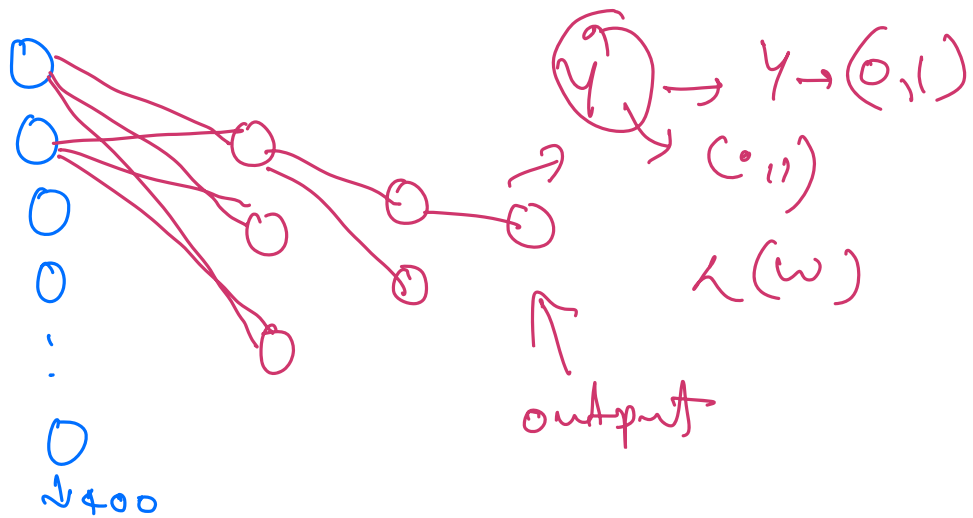


20x20





FC  $\rightarrow$  flatten



BP  $\rightarrow$  GD  $\Rightarrow$  ada



# Table of Contents – Motivation

## 1. Motivation

## 2. RNNs: Recurrent Neural Networks

### 2.1 General Introduction ✓

### 2.2 Vanilla RNNs ✓

### 2.3 Training of RNNs ✓

### 2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow ✓

## 3. LSTM: Long Short-Term Memory

### 3.1 Motivation ✓

### 3.2 LSTM Architecture ✓

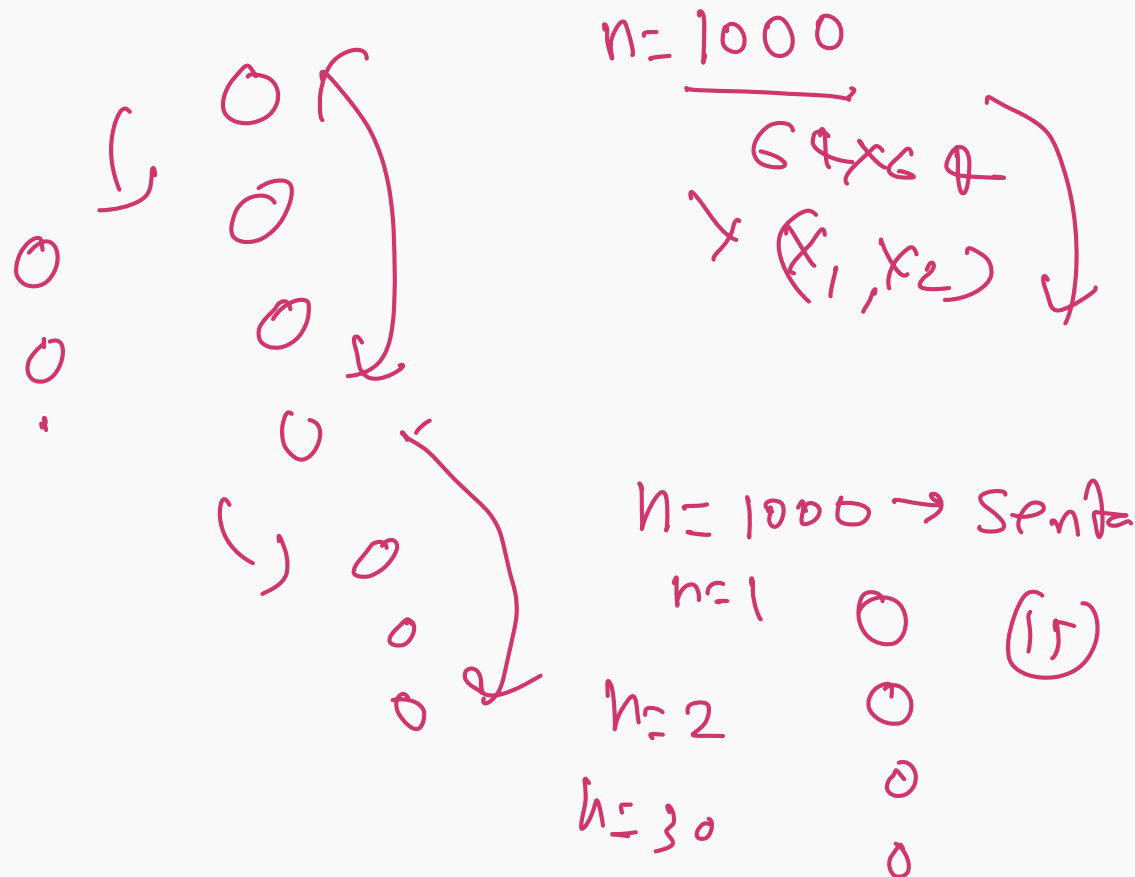
### 3.3 Implementing LSTMs in Tensorflow ✓

# Motivation

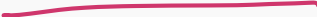
---

# Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs.



# Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs. 
- Some tasks, such as Speech Recognition or Time-series Prediction, require a system to store and use context information.   

# Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs.
- Some tasks, such as Speech Recognition or Time-series Prediction, require a system to store and use context information.
- **Example:** Output YES if the number of 1s is even, else NO:

# Motivation

- **Not all problems** can be converted into one with fixed-length inputs and outputs.
- Some tasks, such as Speech Recognition or Time-series Prediction, require a system to store and use context information.
- **Example:** Output YES if the number of 1s is even, else NO:
  - 1000010101 → YES

# Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
  - 1000010101 → **YES**
  - 100011 → **NO**  


# Motivation

- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
  - 1000010101 → **YES**
  - 100011 → **NO**
- It is **hard or impossible** to choose a fixed context window:

# Motivation

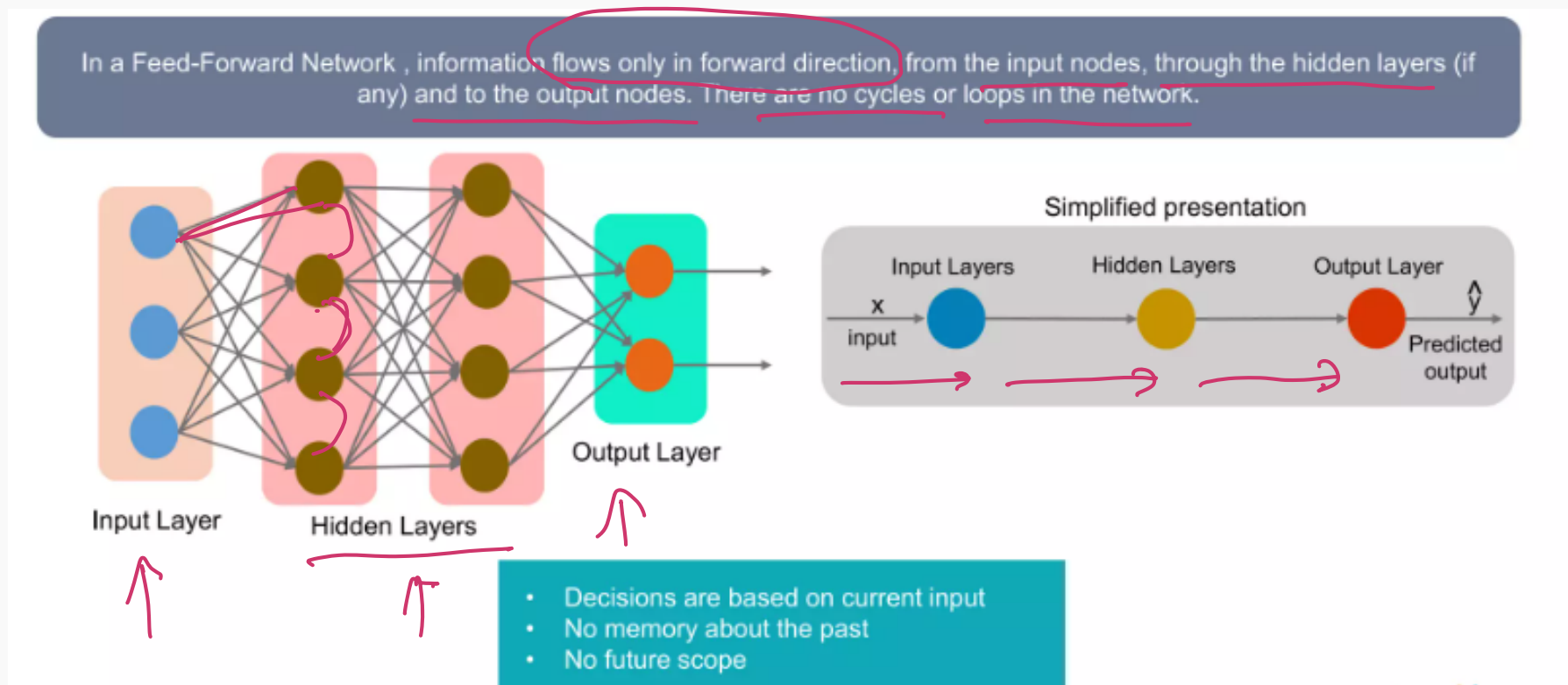
- **Not all problems** can be converted into one with **fixed-length inputs and outputs**.
- Some tasks, such as **Speech Recognition** or **Time-series Prediction**, require a system to **store and use context information**.
- **Example:** Output **YES** if the number of 1s is even, else **NO**:
  - 1000010101 → **YES**
  - 100011 → **NO**
- It is **hard or impossible** to choose a fixed context window:
  - There can always be a new sample longer than anything seen before.

# Recap: Feedforward Neural Networks (FNNs)

Generally, when we talk about simple neural networks, they typically mean feedforward neural networks (FNNs) — especially the multi-layer perceptron (MLP) type. These are the foundational architectures used in many early machine learning models.

# Recap: Feedforward Neural Networks (FNNs)

Generally, when we talk about **simple neural networks**, they typically mean **feedforward neural networks (FNNs)** — especially the multi-layer perceptron (MLP) type. These are the foundational architectures used in many early machine learning models.



# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
  - Natural Language: “The cat sat”  $\neq$  “Sat the cat”

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
  - Natural Language: “The cat sat”  $\neq$  “Sat the cat”
  - Time-Series: Predictions require trends from earlier time points

# Limitations of FNNs

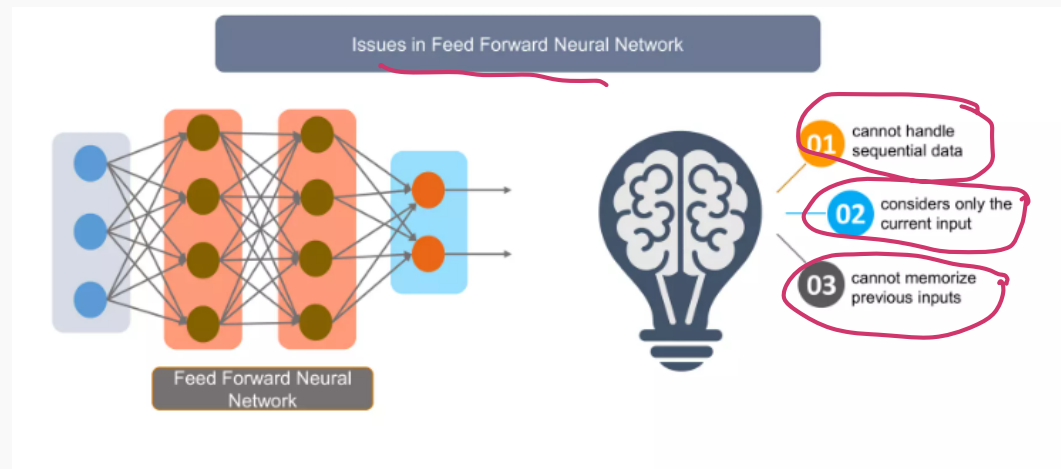
- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
  - Natural Language: “The cat sat”  $\neq$  “Sat the cat”
  - Time-Series: Predictions require trends from earlier time points
  - Speech/Audio: Context and timing are crucial

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
  - Natural Language: “The cat sat”  $\neq$  “Sat the cat”
  - Time-Series: Predictions require trends from earlier time points
  - Speech/Audio: Context and timing are crucial

# Limitations of FNNs

- FNNs treat each input as independent — they lack any notion of order or context.
- No mechanism to retain information about previous inputs or time steps.
- Fails on tasks where input sequence matters, such as:
  - Natural Language: “The cat sat”  $\neq$  “Sat the cat”
  - Time-Series: Predictions require trends from earlier time points
  - Speech/Audio: Context and timing are crucial



FNNs process each input independently: context is lost.

# Motivation for RNNs and LSTMs

We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

# Motivation for RNNs and LSTMs

We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

## Recurrent Neural Networks (RNNs):

- Introduce a feedback loop — current output depends on past hidden states
- Suitable for sequence modeling tasks (language, time-series)
- Struggle with long-term dependencies due to vanishing gradients

# Motivation for RNNs and LSTMs

## We need models that can:

- Learn from sequences — where order and context matter
- Retain and reuse past information — like memory
- Adjust their understanding as more input is seen

## Recurrent Neural Networks (RNNs):

- Introduce a feedback loop — current output depends on past hidden states
- Suitable for sequence modeling tasks (language, time-series)
- Struggle with **long-term dependencies** due to vanishing gradients

## Long Short-Term Memory (LSTM):

- Enhances RNNs with memory cells and gating mechanisms
- Capable of learning long-range dependencies effectively
- Avoids vanishing gradient issues via internal memory regulation

# RNNs: Recurrent Neural Networks

---

# Agenda

## 2. RNNs: Recurrent Neural Networks

### 2.1 General Introduction

### 2.2 Vanilla RNNs

### 2.3 Training of RNNs

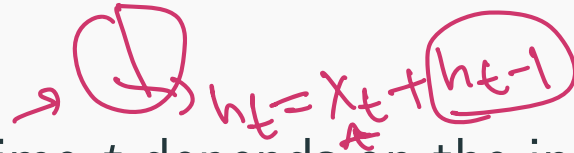
### 2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.



A handwritten diagram in pink ink. On the left, a circle represents an RNN cell with an input arrow on the left and an output arrow on the right. To its right is the equation  $h_t = x_t + h_{t-1}$ . The term  $h_{t-1}$  is circled, and an arrow points from this circle back to the input of the cell, illustrating the recurrent connection.

- The output at time  $t$  depends on the input at  $t$  and the hidden state from  $t - 1$ . That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

- The output at time  $t$  depends on the input at  $t$  and the hidden state from  $t - 1$ . That means: RNNs take the previous output or hidden states as part of the input at each time step.
- The composite input at time  $t$  carries historical information from earlier time steps ( $T < t$ ).

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.

- The output at time  $t$  depends on the input at  $t$  and the hidden state from  $t - 1$ . That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.
- The composite input at time  $t$  carries **historical information** from earlier time steps ( $T < t$ ).
- RNNs are powerful because their intermediate values (also called **hidden states**) can store information about past inputs for a time that is **not fixed a priori**.

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling sequential data. They maintain an internal memory to capture temporal dynamics.

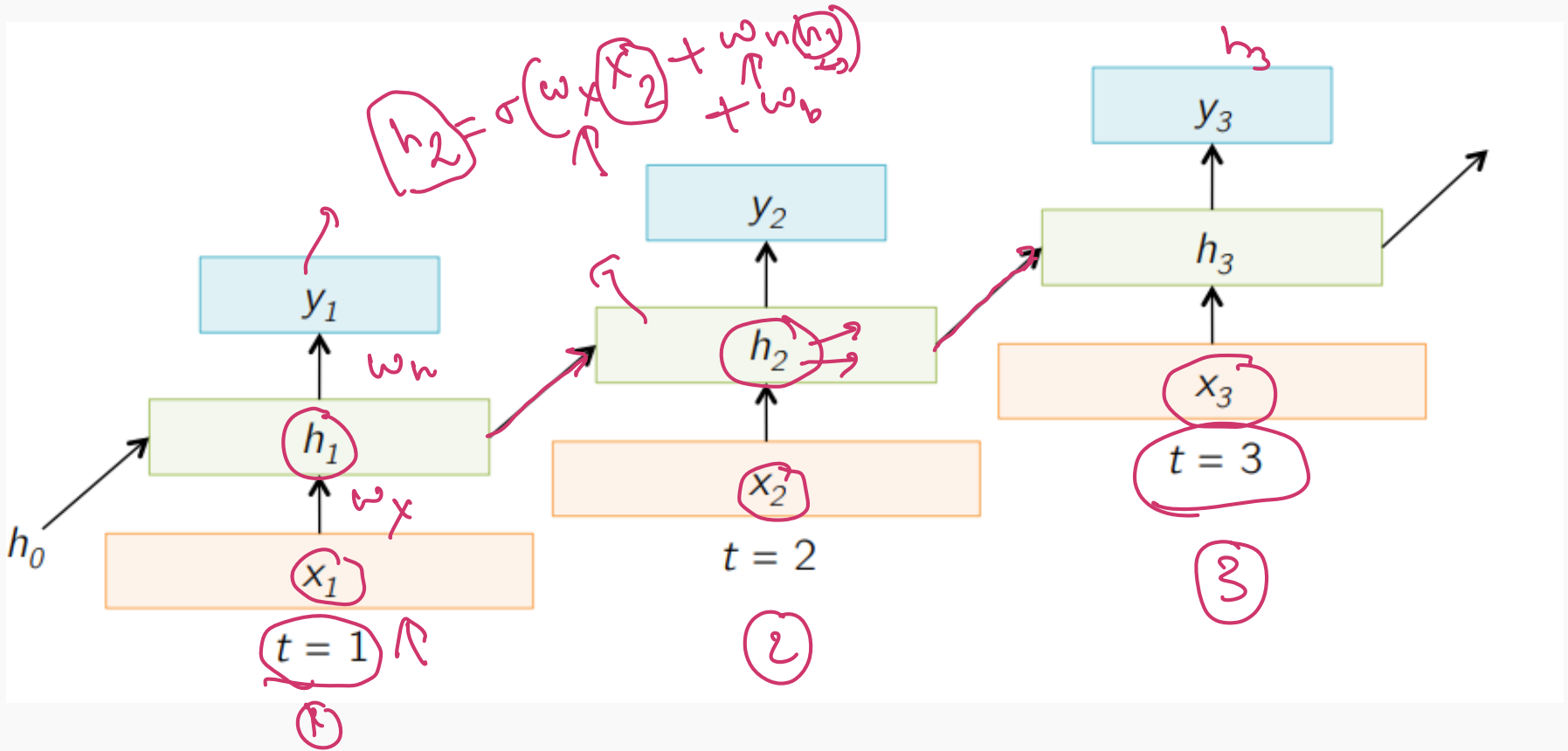
- The output at time  $t$  depends on the input at  $t$  and the hidden state from  $t - 1$ . That means: RNNs take the previous output or hidden states as part of the input at each time step.
- The composite input at time  $t$  carries historical information from earlier time steps ( $T < t$ ).
- RNNs are powerful because their intermediate values (also called hidden states) can store information about past inputs for a time that is not fixed a priori.
- Ideal for problems where context or order matters

# What is a Recurrent Neural Networks (RNNs)?

RNNs are a class of neural networks designed for modeling **sequential data**. They maintain an internal memory to capture temporal dynamics.

- The output at time  $t$  depends on the input at  $t$  and the hidden state from  $t - 1$ . That means: RNNs take the **previous output** or **hidden states** as part of the input at each time step.
- The composite input at time  $t$  carries **historical information** from earlier time steps ( $T < t$ ).
- RNNs are powerful because their intermediate values (also called **hidden states**) can **store information** about past inputs for a time that is **not fixed a priori**.
- Ideal for problems where **context or order matters**
- Used in language modeling, speech, and time-series tasks

# Sample RNNs



Where

- $h_t$  is the hidden state at time  $t$
- $x_t$  is the input vector at time  $t$

# Agenda

## 2. RNNs: Recurrent Neural Networks

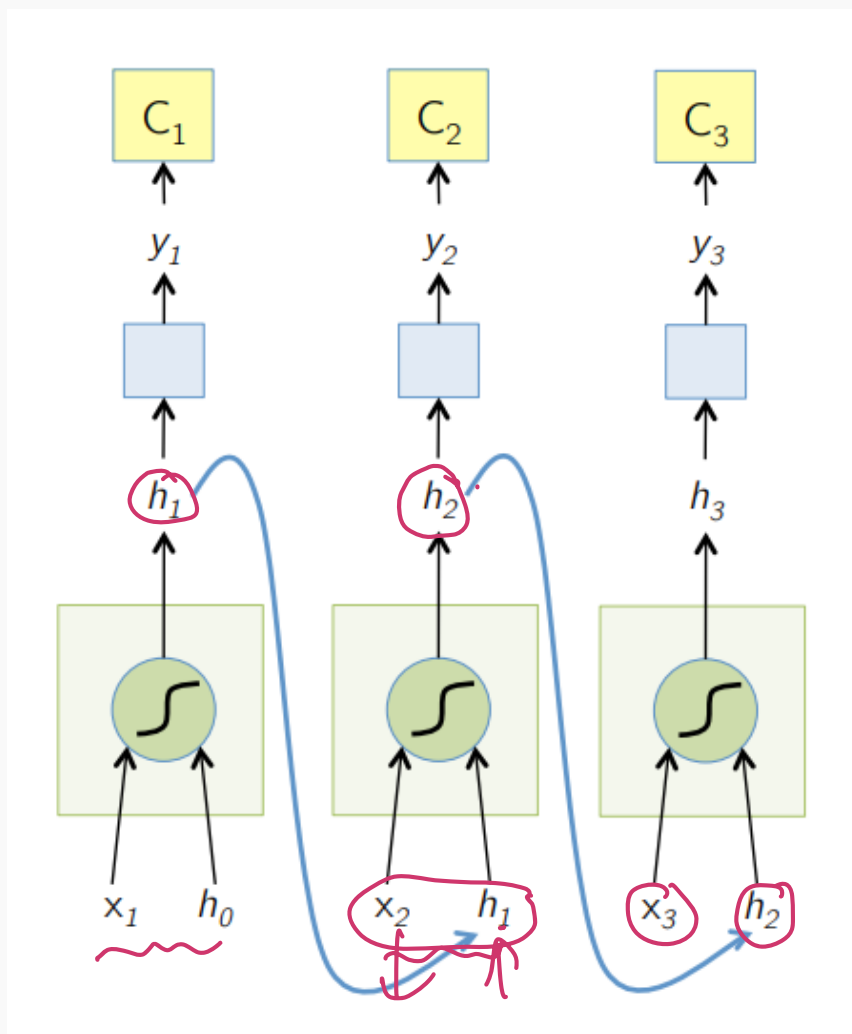
### 2.1 General Introduction

### 2.2 Vanilla RNNs

### 2.3 Training of RNNs

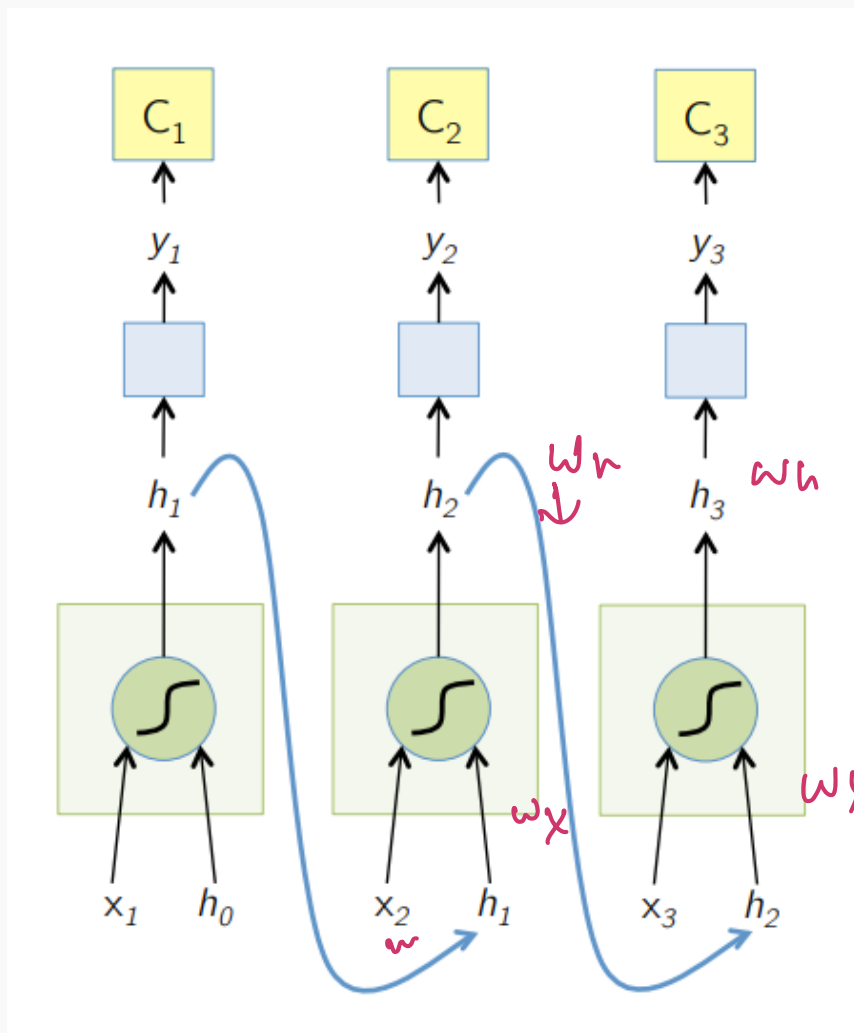
### 2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

# RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequence length.

# RNNs Forward Steps for (for Vanilla RNNs)



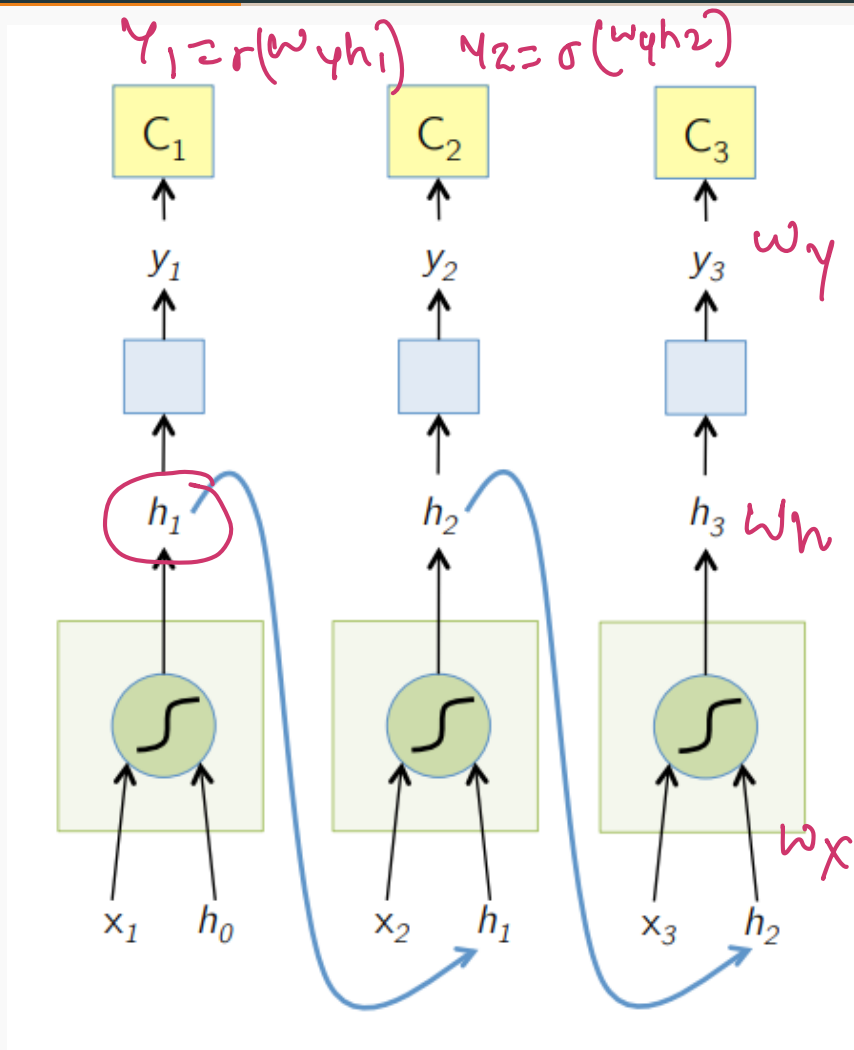
- The basic RNN cell updates as:

$$h_t = \tanh(\underbrace{W_h}_{w_h} h_{t-1} + \underbrace{W_x}_{w_x} x_t + \underbrace{b}_{w_b})$$

where  $\underbrace{W_h}_{w_h}$ ,  $\underbrace{W_x}_{w_x}$ , and  $\underbrace{b}_{w_b}$  are learnable parameters.

An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

# RNNs Forward Steps for (for Vanilla RNNs)



- The basic RNN cell updates as:

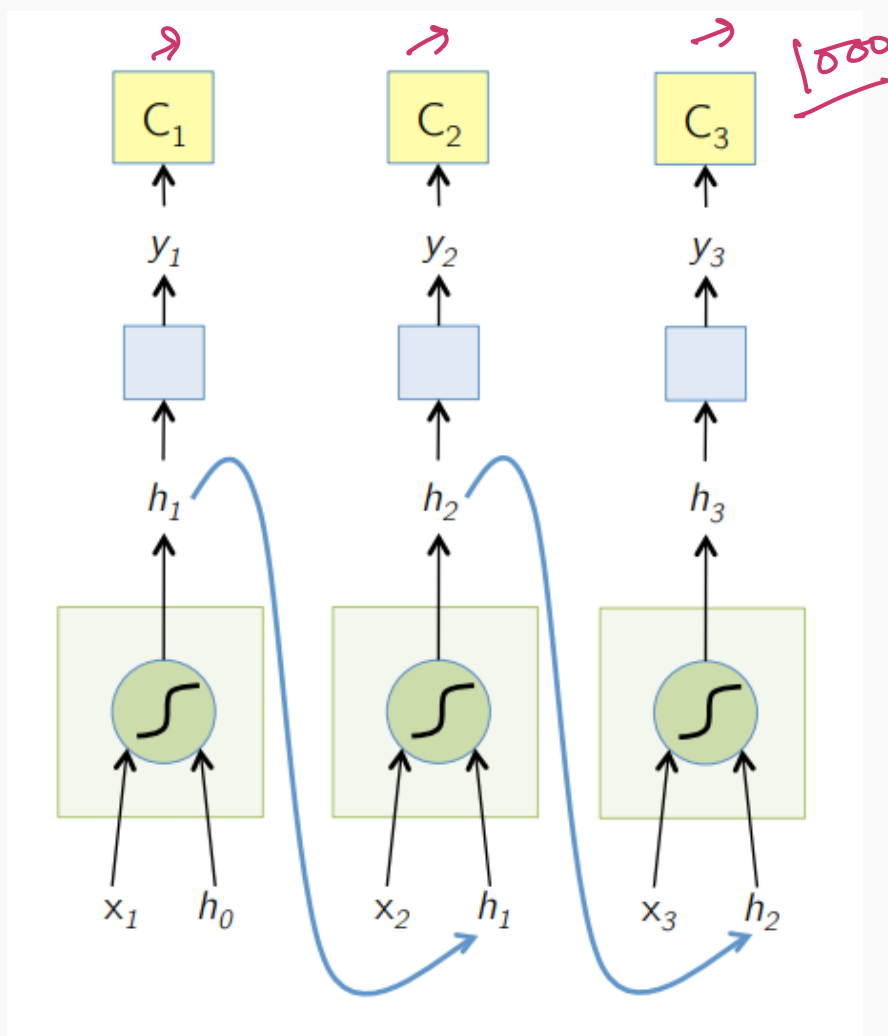
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where  $W_h$ ,  $W_x$ , and  $b$  are learnable parameters.

- The output  $y_t = \sigma(W_y h_t)$ , where  $\sigma(\cdot)$  is the activation function for the output layer

An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

# RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

- The basic RNN cell updates as:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

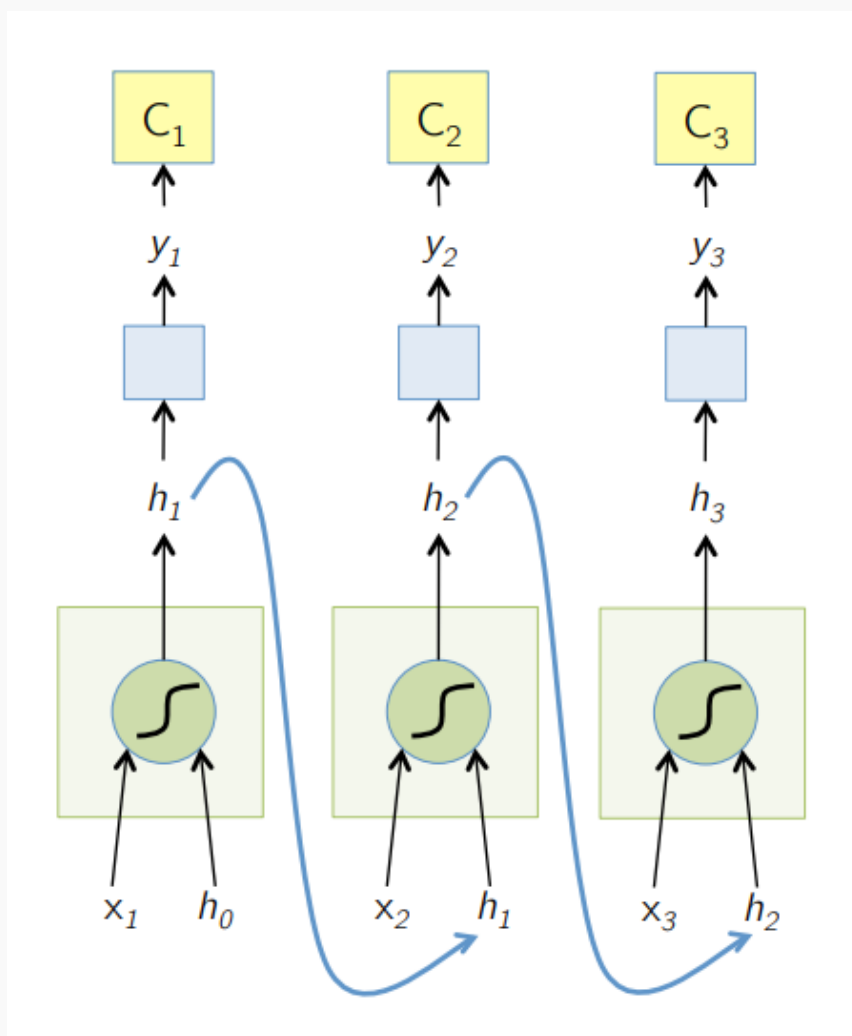
where  $W_h$ ,  $W_x$ , and  $b$  are learnable parameters.

- The output  $y_t = \sigma(W_y h_t)$ , where  $\sigma(\cdot)$  is the activation function for the output layer
- $C_t = \text{Loss}(y_t, \text{GT}_t)$  is the loss calculated at time  $t$

$$L(W, C) = \frac{1}{n} \sum_{i=1}^n C_i$$

$$W = (W_h, W_x, W_y, b) \quad C_i = (y_t - \hat{y}_t)^2$$

# RNNs Forward Steps for (for Vanilla RNNs)



An RNN processes input sequences one step at a time. The same RNN cell (with shared weights) is reused across all time steps. This makes the RNN look like a deep network with depth equal to the sequencelength.

- The basic RNN cell updates as:  

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

where  $W_h$ ,  $W_x$ , and  $b$  are learnable parameters.

Handwritten notes:  $\max(0, x) \rightarrow \begin{cases} 0 & \text{if } x \leq 0 \\ x & \text{if } x > 0 \end{cases}$ ,  $h_1 = h_0$ ,  $h_2 = h_1$ ,  $h_3 = h_2$ ,  $\text{relu}(x) = \max(0, x) \Rightarrow \text{big value}$ ,  $\in (-1, 1)$
- The output  $y_t = \sigma(W_y h_t)$ , where  $\sigma(\cdot)$  is the activation function for the output layer
- $C_t = \text{Loss}(y_t, \text{GT}_t)$  is the loss calculated at time  $t$
- Note that the weights  $W_h$ ,  $W_x$ ,  $W_y$  and biases  $b$  are shared over time

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - *“The food was really good.”*  $\Rightarrow$  **Positive**

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - *“The food was really good.”* ⇒ **Positive**
  - *“The chicken crossed the road because it was uncooked.”* ⇒ **Negative**

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - *“The food was really good.”* ⇒ **Positive**
  - *“The chicken crossed the road because it was uncooked.”* ⇒ **Negative**
  - *“Absolutely loved the cinematography and acting.”* ⇒ **Positive**

# RNNs application: Sentiment Classification

- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - “*The food was really good.*” ⇒ **Positive**
  - “*The chicken crossed the road because it was uncooked.*” ⇒ **Negative**
  - “*Absolutely loved the cinematography and acting.*” ⇒ **Positive**
  - “*The plot was dull and completely predictable.*” ⇒ **Negative**

# RNNs application: Sentiment Classification

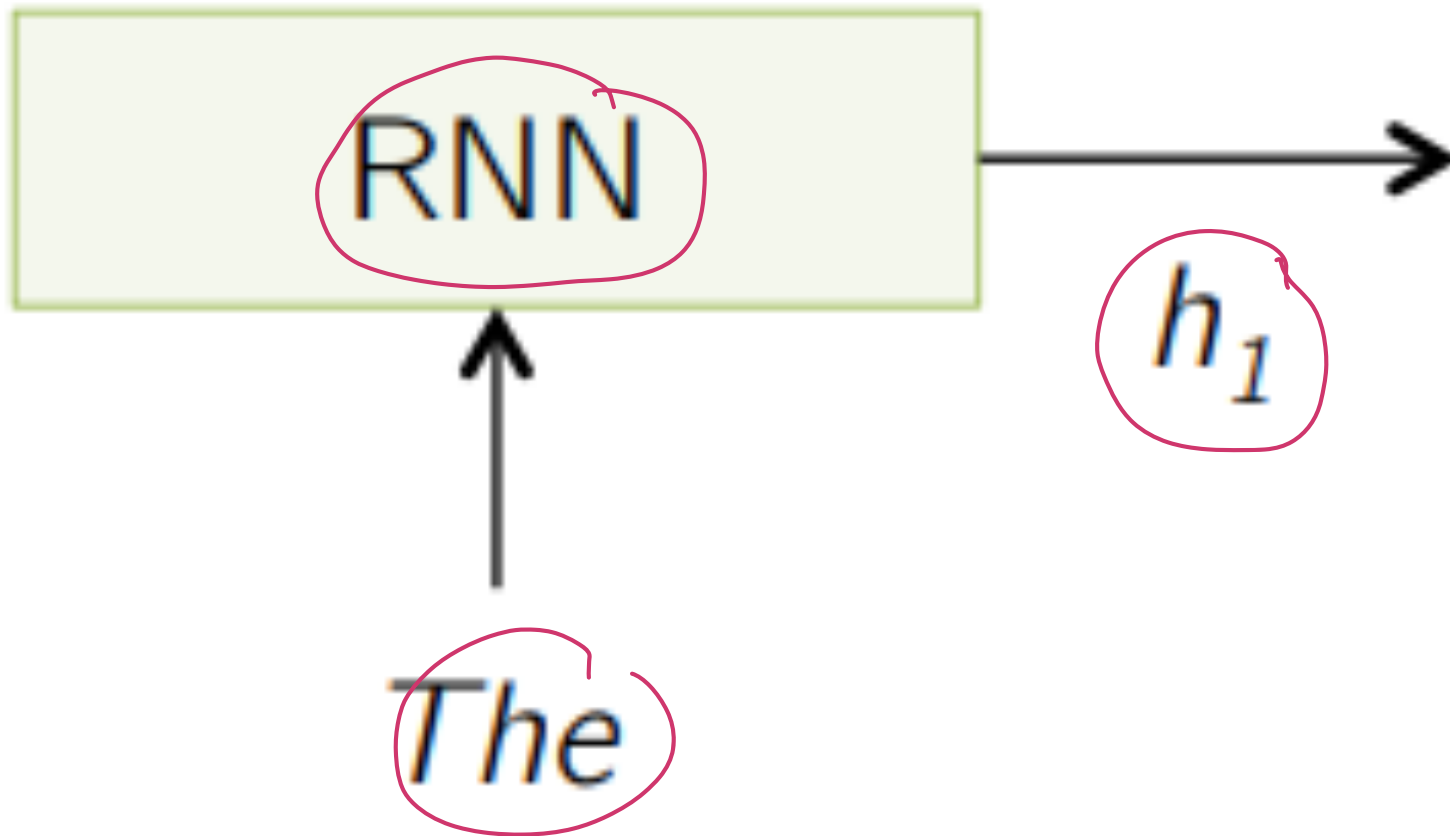
- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - “The food was really good.” ⇒ **Positive**
  - “The chicken crossed the road because it was uncooked.” ⇒ **Negative**
  - “Absolutely loved the cinematography and acting.” ⇒ **Positive**
  - “The plot was dull and completely predictable.” ⇒ **Negative**
  - “Would definitely come here again!” ⇒ **Positive**

# RNNs application: Sentiment Classification

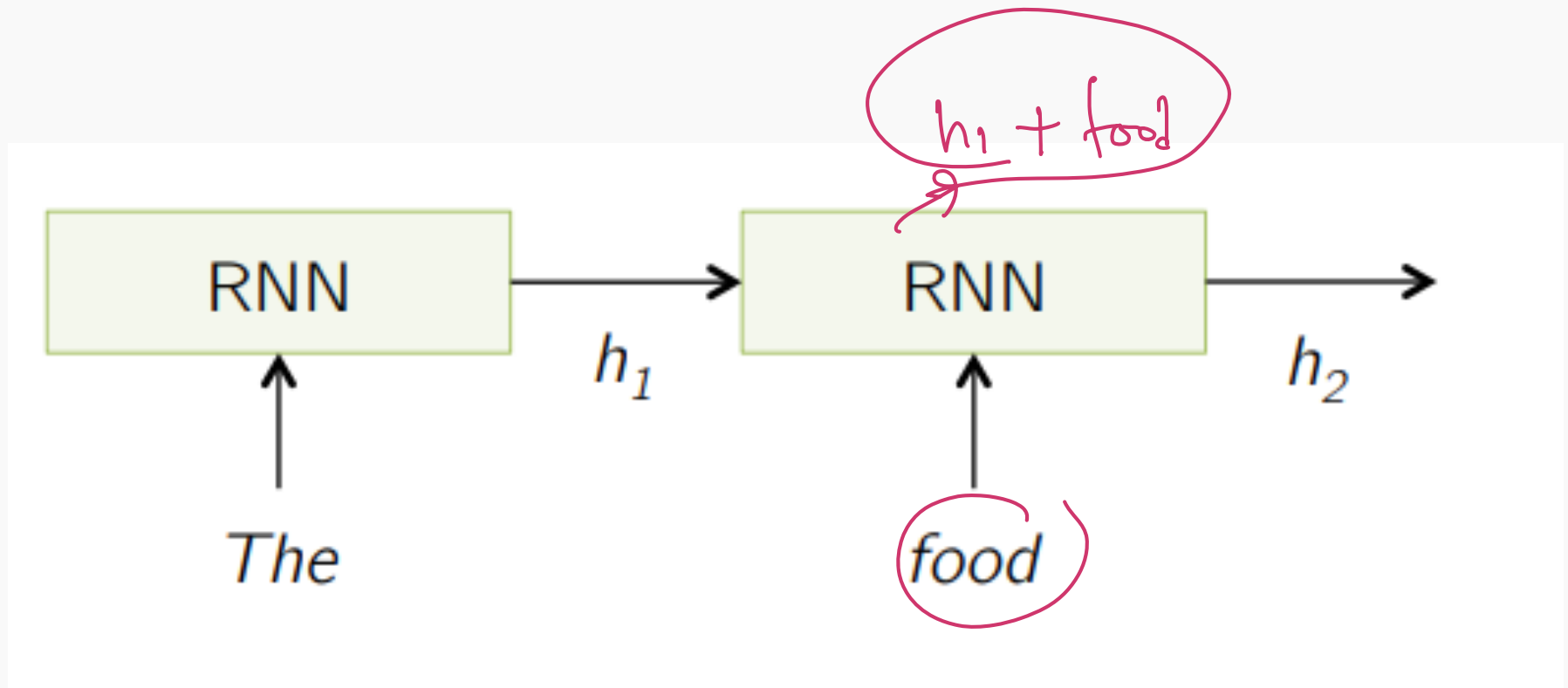
- **Goal:** Classify a review (e.g., from [Yelp!](#), [IMDB](#), etc.) as **positive** or **negative**.
- **Inputs:** Multiple words forming one or more sentences.
- **Outputs:** Binary classification — Positive / Negative.
- **Examples:**
  - *“The food was really good.”* ⇒ **Positive**
  - *“The chicken crossed the road because it was uncooked.”* ⇒ **Negative**
  - *“Absolutely loved the cinematography and acting.”* ⇒ **Positive**
  - *“The plot was dull and completely predictable.”* ⇒ **Negative**
  - *“Would definitely come here again!”* ⇒ **Positive**
  - *“Terrible service and cold food.”* ⇒ **Negative**

# Sentiment Classification I

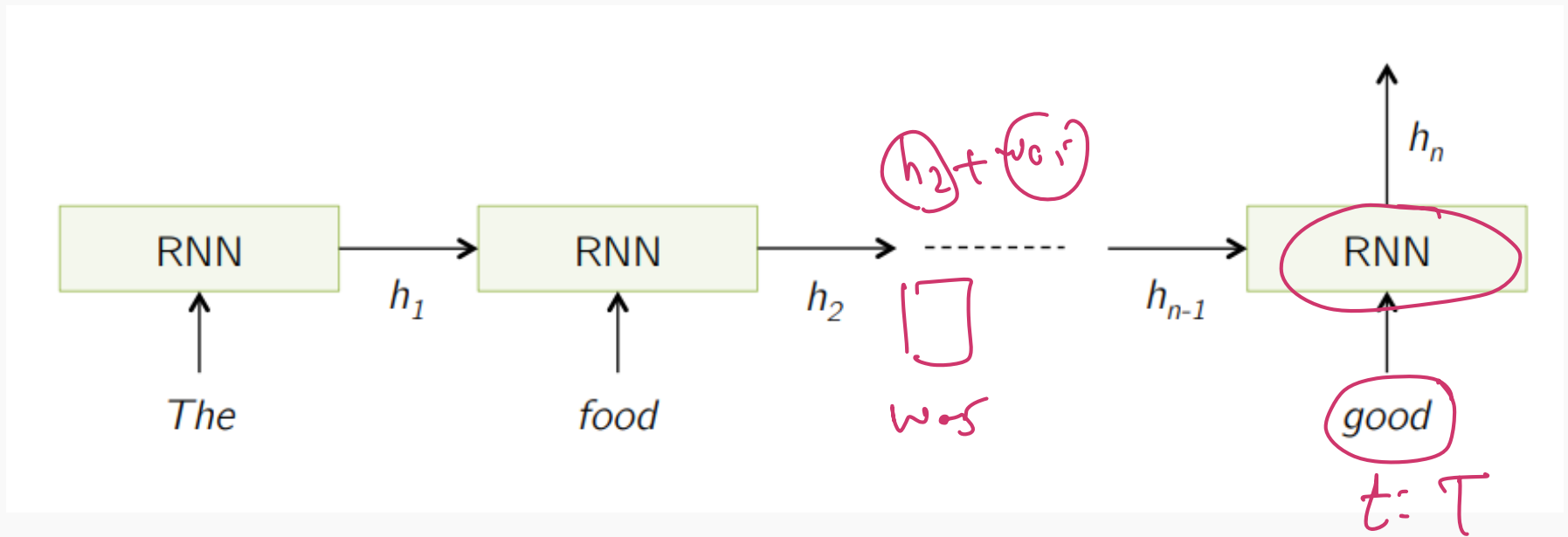
Consider the input: "The food was really good"



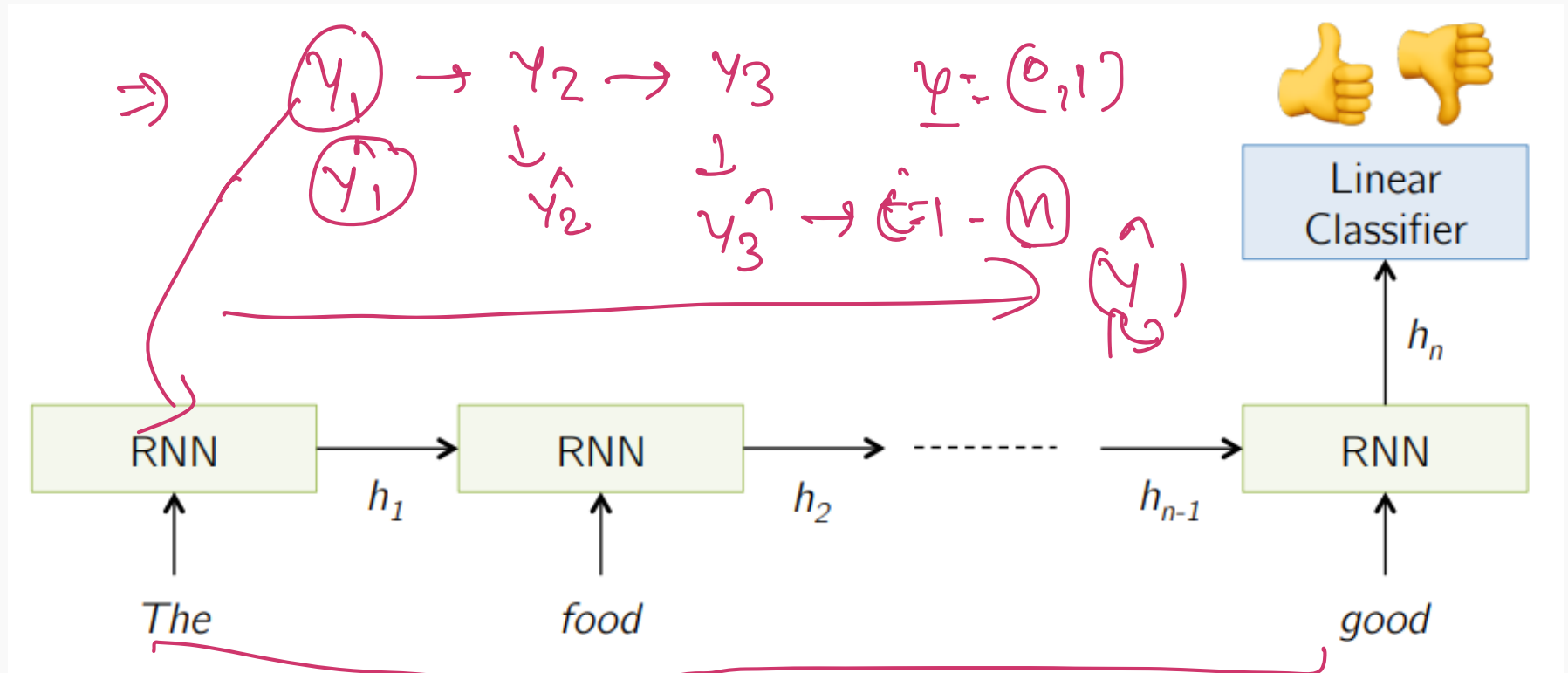
# Sentiment Classification II



# Sentiment Classification III

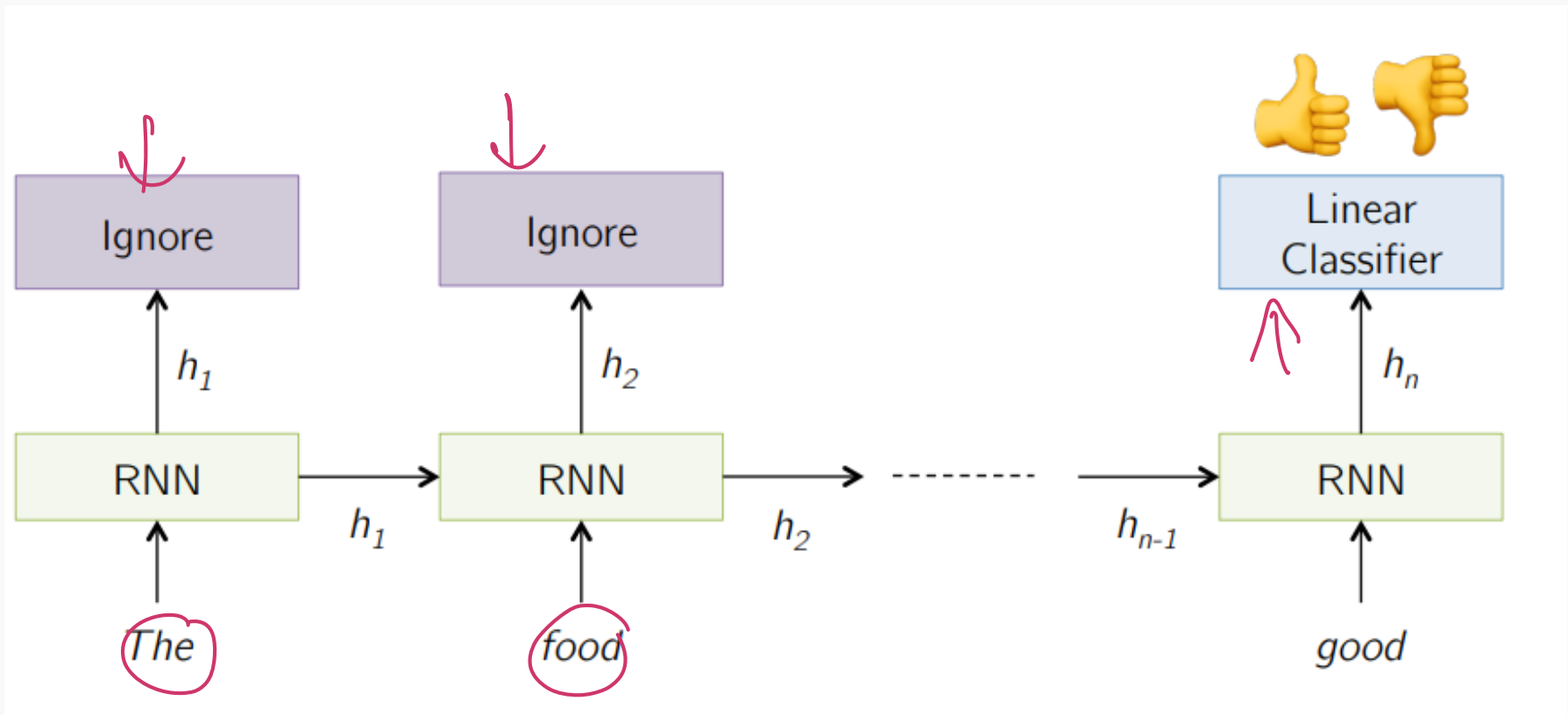


# Sentiment Classification IV



$$\Rightarrow L(w) =$$

# Sentiment Classification V



# Agenda

## 2. RNNs: Recurrent Neural Networks

### 2.1 General Introduction

### 2.2 Vanilla RNNs

### 2.3 Training of RNNs

### 2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the forward RNN steps:
  - Calculation at  $t$ -th hidden state

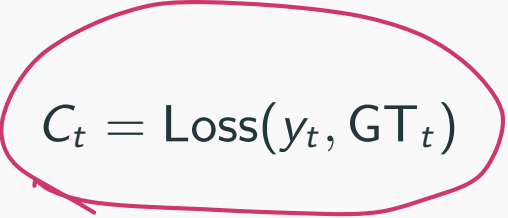
$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:
  - Calculation at  $t$ -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time  $t$ :


$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:
  - Calculation at  $t$ -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time  $t$ :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across  $T$  time steps:

$$C = \sum_{t=1}^T C_t$$

Handwritten notes:  $C_T \rightarrow C = C_T$  and  $C$  with arrows pointing to the summation symbol.

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:
  - Calculation at  $t$ -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time  $t$ :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across  $T$  time steps:

$$C = \sum_{t=1}^T C_t$$

- **The Backpropagation Step:**

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:
  - Calculation at  $t$ -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time  $t$ :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across  $T$  time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

- Compute gradients  $\frac{\partial C}{\partial W_h}$ ,  $\frac{\partial C}{\partial W_x}$ ,  $\frac{\partial C}{\partial W_y}$ ,  $\frac{\partial C}{\partial b}$

# RNNs Backward Steps: Backpropagation Through Time (BPTT)

- Recall the **forward RNN** steps:

- Calculation at  $t$ -th hidden state

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b) \quad \text{and} \quad y_t = \sigma(W_y h_t)$$

- The loss at time  $t$ :

$$C_t = \text{Loss}(y_t, \text{GT}_t)$$

- Total loss across  $T$  time steps:

$$C = \sum_{t=1}^T C_t$$

- The Backpropagation Step:**

- Compute gradients  $\frac{\partial C}{\partial W_h}$ ,  $\frac{\partial C}{\partial W_x}$ ,  $\frac{\partial C}{\partial W_y}$ ,  $\frac{\partial C}{\partial b}$
- Requires unrolling the network over time and applying the chain rule recursively

# Gradient Flow Through Time

- Gradients are propagated backward from time  $T$  to  $t = 1$ .

# Gradient Flow Through Time

- Gradients are propagated backward from time  $T$  to  $t = 1$ .
- Chain rule applied recursively:

$$\frac{\partial C}{\partial W_h} = \sum_{t=1}^T \frac{\partial C_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h}$$

# Gradient Flow Through Time

- Gradients are propagated backward from time  $T$  to  $t = 1$ .
- Chain rule applied recursively:

$$\frac{\partial C}{\partial W_h} = \sum_{t=1}^T \frac{\partial C_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h}$$

- But  $\frac{\partial h_t}{\partial W_h}$  depends on  $h_{t-1}$ , which depends on  $h_{t-2}$ , and so on.

# Gradient Flow Through Time

- Gradients are propagated backward from time  $T$  to  $t = 1$ .
- Chain rule applied recursively:

$$\frac{\partial C}{\partial W_h} = \sum_{t=1}^T \frac{\partial C_t}{\partial h_t} \cdot \frac{\partial h_t}{\partial W_h}$$

- But  $\frac{\partial h_t}{\partial W_h}$  depends on  $h_{t-1}$ , which depends on  $h_{t-2}$ , and so on.
- So gradients accumulate via:

$$\frac{\partial C_t}{\partial h_k} = \frac{\partial C_t}{\partial h_t} \cdot \prod_{j=k+1}^t \frac{\partial h_j}{\partial h_{j-1}}, \quad \text{for } k < t$$

and leads to numerical instability during training

$w_h, w_r, w_x, b$

# Mitigating Exploding Gradients

- **Gradient Clipping:**

$$\text{If } \|\nabla C\| > \tau, \quad \nabla C \leftarrow \tau \cdot \frac{\nabla C}{\|\nabla C\|}$$

# Mitigating Exploding Gradients

- **Gradient Clipping:**

$$\text{If } \|\nabla C\| > \tau, \quad \nabla C \leftarrow \tau \cdot \frac{\nabla C}{\|\nabla C\|}$$

Handwritten notes illustrating gradient clipping:

$$\omega^{(t+1)} = \omega^{(t)} - h \nabla C(\omega^t)$$

When the step size  $h$  is too large (e.g.,  $h=100$ ), the update becomes:

$$\omega^{(t+1)} = \omega^{(t)} - 100h$$

or

$$\omega^{(t+1)} = \omega^{(t)} - h$$

which leads to values like:

$$\omega^{(t+1)} = \omega^{(t)} - 10$$

or

$$\omega^{(t+1)} = \omega^{(t)} - 0.1$$

- **Layer Norm / Batch Norm:** Normalize intermediate states

# Mitigating Exploding Gradients

- **Gradient Clipping:**

$$\text{If } \|\nabla C\| > \tau, \quad \nabla C \leftarrow \tau \cdot \frac{\nabla C}{\|\nabla C\|}$$

- **Layer Norm / Batch Norm:** Normalize intermediate states
- **Architecture Changes:**

# Mitigating Exploding Gradients

- **Gradient Clipping:**

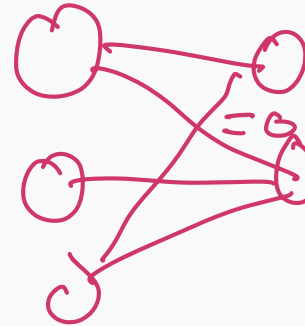
$$\text{If } \|\nabla C\| > \tau, \quad \nabla C \leftarrow \tau \cdot \frac{\nabla C}{\|\nabla C\|}$$

- **Layer Norm / Batch Norm:** Normalize intermediate states

- **Architecture Changes:**

- Use LSTMs or GRUs — designed to handle long-term dependencies better

# Mitigating Exploding Gradients



- **Gradient Clipping:**

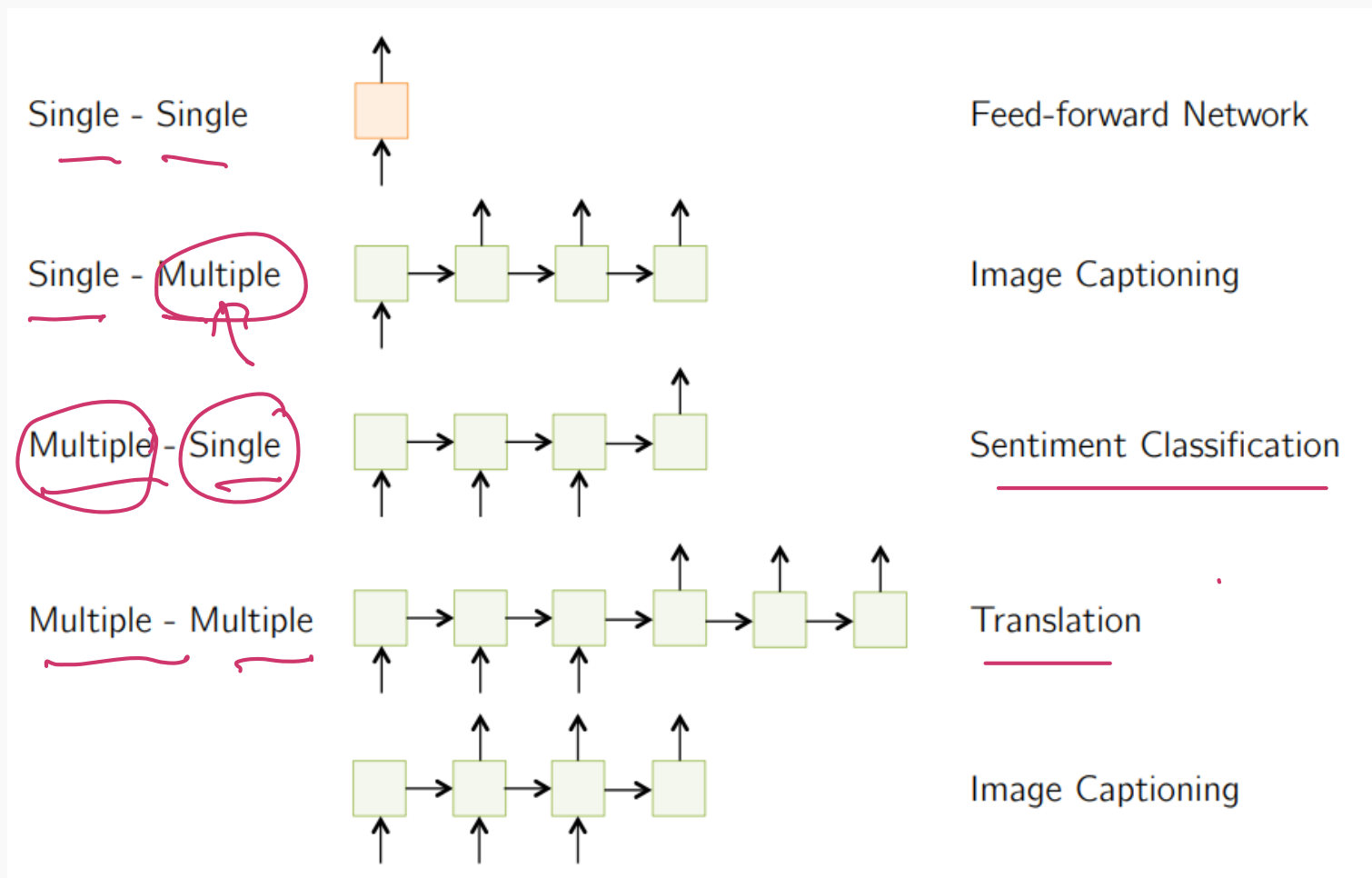
$$\text{If } \|\nabla C\| > \tau, \quad \nabla C \leftarrow \tau \cdot \frac{\nabla C}{\|\nabla C\|}$$

- **Layer Norm / Batch Norm:** Normalize intermediate states

- **Architecture Changes:**

- Use LSTMs or GRUs — designed to handle long-term dependencies better
- Residual (skip) connections (in some variants)

# Input-Output Scenarios in RNNs



We might deliberately choose to frame our problem as a particular input-output scenario for ease of training or better performance.

# Limitations of RNNs

- Vanishing gradients — difficult to learn long-range dependencies
- Exploding gradients — unstable training
- Difficulty retaining information across long sequences
- Struggles with tasks requiring memory over many steps

These limitations lead to poor performance on tasks like language modeling and sequence prediction.

# Agenda

## 2. RNNs: Recurrent Neural Networks

### 2.1 General Introduction

### 2.2 Vanilla RNNs

### 2.3 Training of RNNs

### 2.4 Implementing simple RNNs (Vanilla RNNs) in Tensorflow

# Implementing RNNs in TensorFlow

We demonstrate how to build a simple RNN using `tf.keras` to perform time series prediction. Open the Google Colab notebook from the Link:

[https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module4/code/NNs\\_introduction\\_to\\_RNN\\_GColab.ipynb](https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module4/code/NNs_introduction_to_RNN_GColab.ipynb)

# Step 1: Prepare the Data

- Use a univariate or multivariate time series
- Convert it to input/output sequences using sliding windows

```
1 import numpy as np
2
3 # Example: univariate time series
4 series = np.sin(np.linspace(0, 100, 1000))
5
6 # Prepare X, y sequences
7 def create_sequences(series, window):
8     X, y = [], []
9     for i in range(len(series) - window):
10         X.append(series[i:i+window])
11         y.append(series[i+window])
12     return np.array(X), np.array(y)
13
14 window = 20
15 X, y = create_sequences(series, window)
16 X = X[..., np.newaxis] # shape: (samples, timesteps,
    features=1)
```

## Step 2: Build the RNN Model

- Use SimpleRNN for sequential modeling
- Output a single value for each sequence

```
1 model = tf.keras.Sequential([  
2     tf.keras.layers.SimpleRNN(50, activation='tanh',  
3         input_shape=(X.shape[1], X.shape[2])),  
4     tf.keras.layers.Dense(1)  
5 ])   
6 model.compile(optimizer='adam', loss='mse')  
7 model.summary()
```

## Step 3: Train the RNN Model

- Fit the model to predict future values from past sequences
- Use a validation set to monitor performance

```
1 from sklearn.model_selection import train_test_split
2
3 X_train, X_val, y_train, y_val = train_test_split(
4     X, y, test_size=0.2, shuffle=False)
5
6 model.fit(X_train, y_train,
7           validation_data=(X_val, y_val),
8           epochs=20, batch_size=32)
```

## Step 4: Make Predictions

- Predict the next value(s) in the sequence

```
1 y_pred = model.predict(X_val)
2
3 # Plot predictions
4 import matplotlib.pyplot as plt
5
6 plt.plot(y_val[:100], label='True')
7 plt.plot(y_pred[:100], label='Predicted')
8 plt.legend(); plt.title("Time Series Prediction")
```

# Customization Notes and Summary

- **Customization Notes**

- **Initial states:** Use `initial_state=` argument in `SimpleRNN()` if needed
- **Stacked RNNs:** Set `return_sequences=True` in lower layers
- **Dropout:** `dropout=` and `recurrent_dropout=` to regularize
- **Cell-level API:** Use `tf.keras.layers.RNN(cell)` for full customization
- **Gradients:** TensorFlow automatically handles BPTT

- **Summary**

- RNNs are useful for sequence modeling tasks
- TensorFlow provides high-level RNN APIs (`SimpleRNN`, `LSTM`, `GRU`)
- Training requires careful preprocessing and attention to input shape
- For long sequences, consider `LSTMs` or `GRUs` for better stability

# LSTM: Long Short-Term Memory

---

## 3. LSTM: Long Short-Term Memory

### 3.1 Motivation

### 3.2 LSTM Architecture

### 3.3 Implementing LSTMs in Tensorflow

# Why Do We Need LSTM?

Standard RNNs suffer from:

- **Vanishing/exploding gradients**: hard to learn long dependencies
- **Short-term memory**: forgets earlier inputs quickly



# Why Do We Need LSTM?

Standard RNNs suffer from:

- **Vanishing/exploding gradients**: hard to learn long dependencies
- **Short-term memory**: forgets earlier inputs quickly

**LSTM** was introduced to overcome the major shortcomings of traditional RNNs. **LSTM networks** solve this with a unique **cell state** and **gating mechanisms** that control information flow.

# Why Do We Need LSTM?

Standard RNNs suffer from:

- **Vanishing/exploding gradients**: hard to learn long dependencies
- **Short-term memory**: forgets earlier inputs quickly

**LSTM** was introduced to overcome the major shortcomings of traditional RNNs. **LSTM networks** solve this with a unique **cell state** and **gating mechanisms** that control information flow.

**LSTM** can preserve information over **long sequences** and

- Decide what to **remember**
- Decide what to **forget**
- Decide what to **output**

# LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

# LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

- Introduced by Hochreiter and Schmidhuber (1997)

# LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

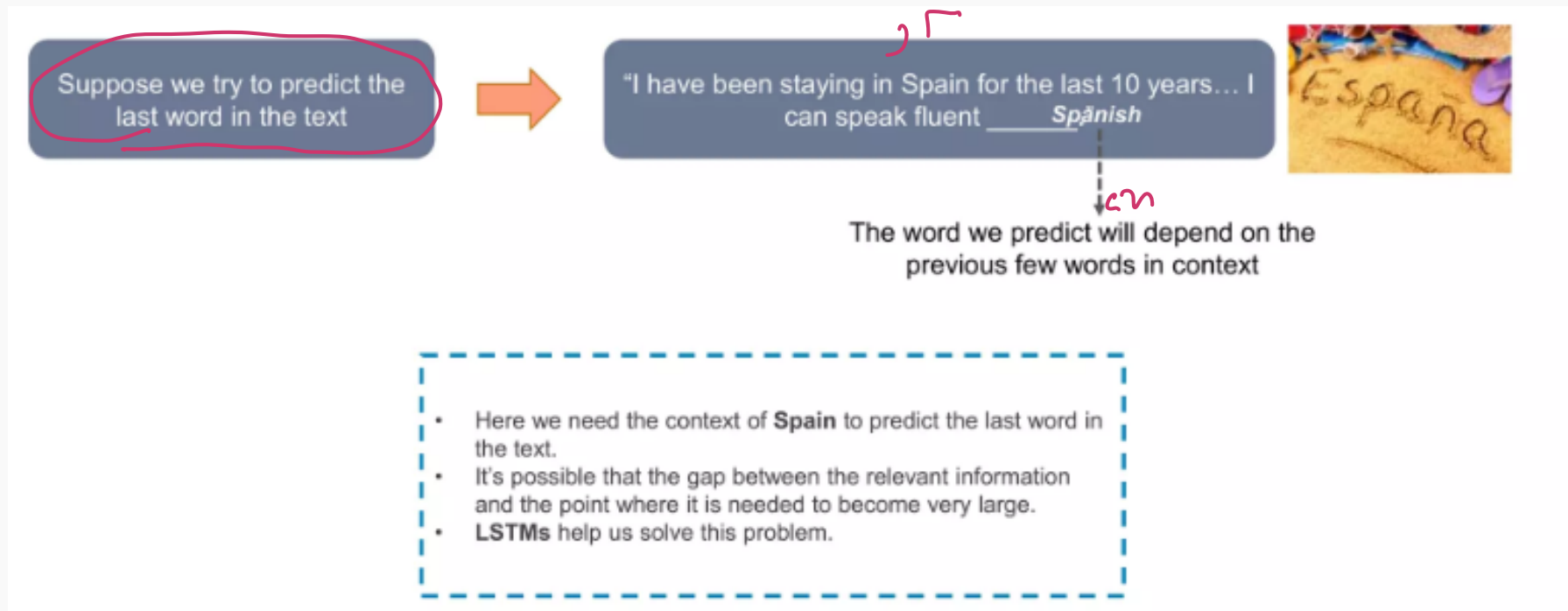
- Introduced by Hochreiter and Schmidhuber (1997)
  - Effective at learning long-range sequences
- 

# LSTM: Long Short-Term Memory

LSTM is a special type of Recurrent Neural Network (RNN) designed to learn long-term dependencies. It addresses the limitations of standard RNNs which struggle with vanishing and exploding gradients.

- Introduced by Hochreiter and Schmidhuber (1997)
- Effective at learning long-range sequences
- Widely used in NLP, speech recognition, and time series forecasting

# Why Long-Term Dependencies is Important?



## 3. LSTM: Long Short-Term Memory

### 3.1 Motivation

### 3.2 LSTM Architecture

### 3.3 Implementing LSTMs in Tensorflow

# LSTM Architecture (1)

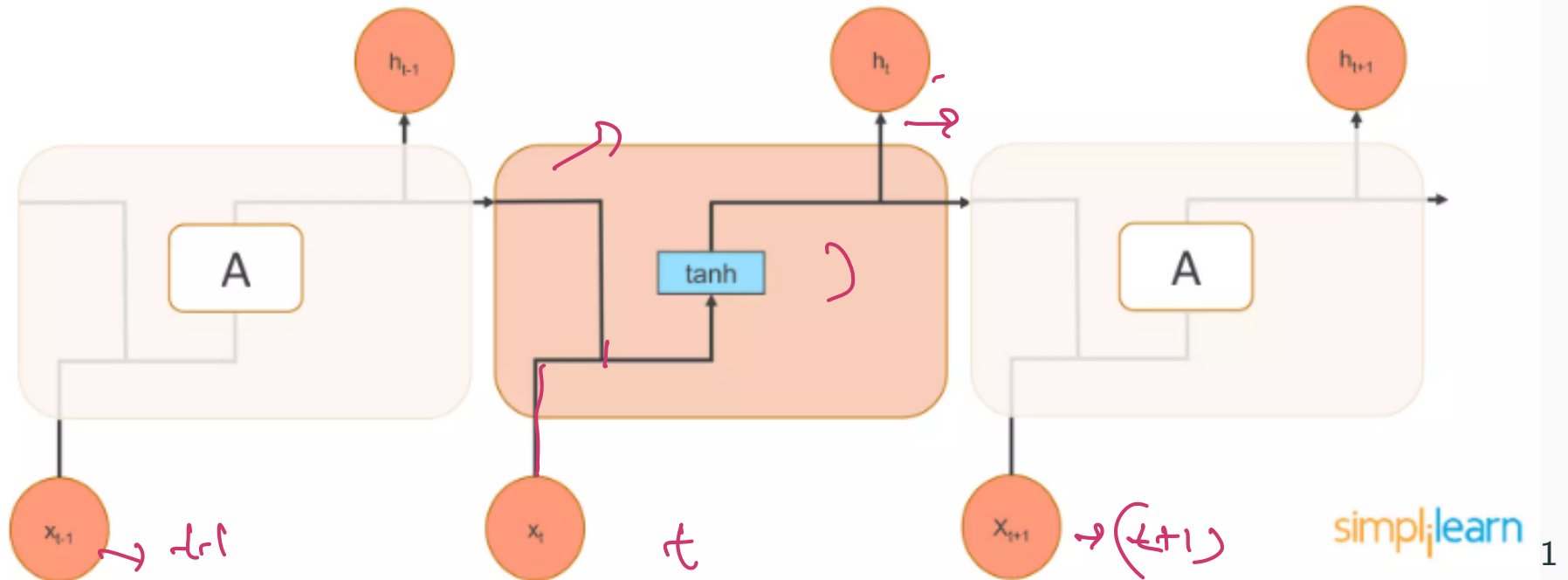
- The key feature is the cell state ( $C_t$ ), which runs through the entire chain with only minor interactions.
- Information is added or removed via gates:
  - Forget gate  $f_t$
  - Input gate  $i_t$
  - Output gate  $o_t$

Hidden state  $h_t$  carries the output at each time step.

# LSTM Architecture (2)

LSTMs are special kind of Recurrent Neural Networks, capable of learning long-term dependencies. Remembering information for long periods of time is their default behavior.

All recurrent neural networks have the form of a chain of repeating modules of neural network. In standard RNNs, this repeating module will have a very simple structure, such as a single tanh layer.

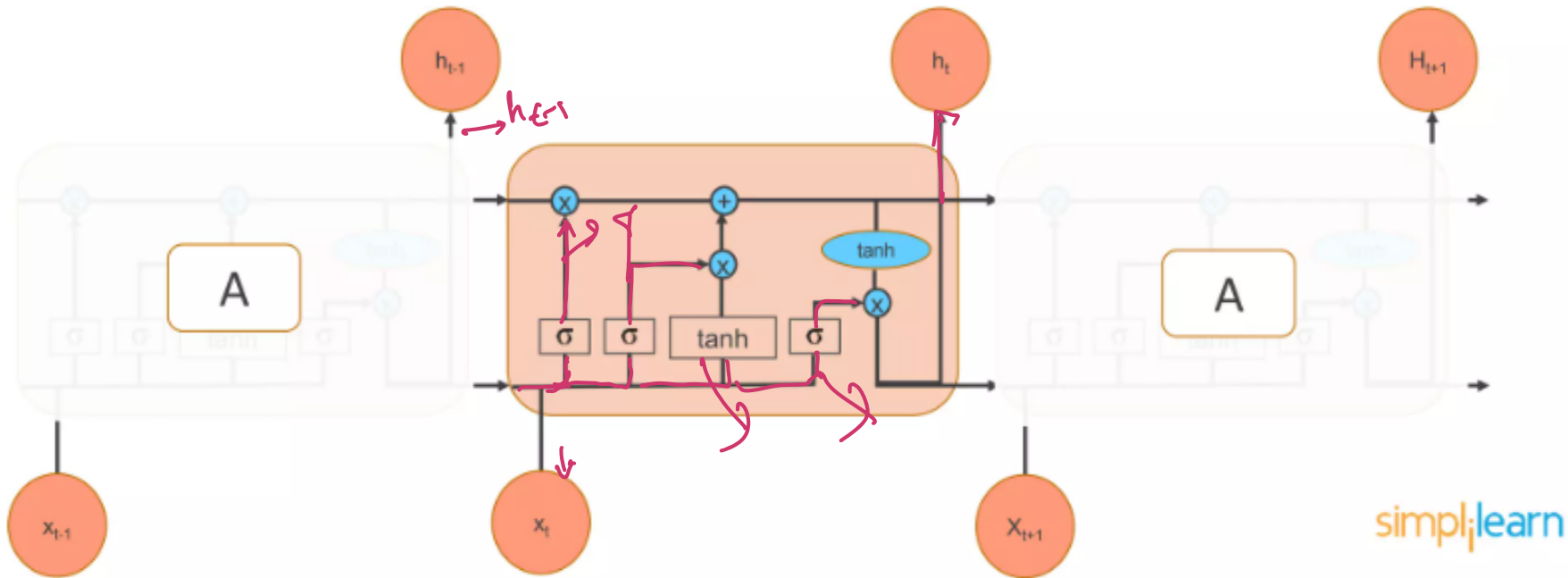


<sup>1</sup> <https://www.simplilearn.com/>

# LSTM Architecture (3)

LSTMs are special kind of Recurrent Neural Networks, capable of learning long-term dependencies. Remembering information for long periods of time is their default behavior.

LSTMs also have a chain like structure, but the repeating module has a different structure. Instead of having a single neural network layer, there are four interacting layers communicating in a very special way.



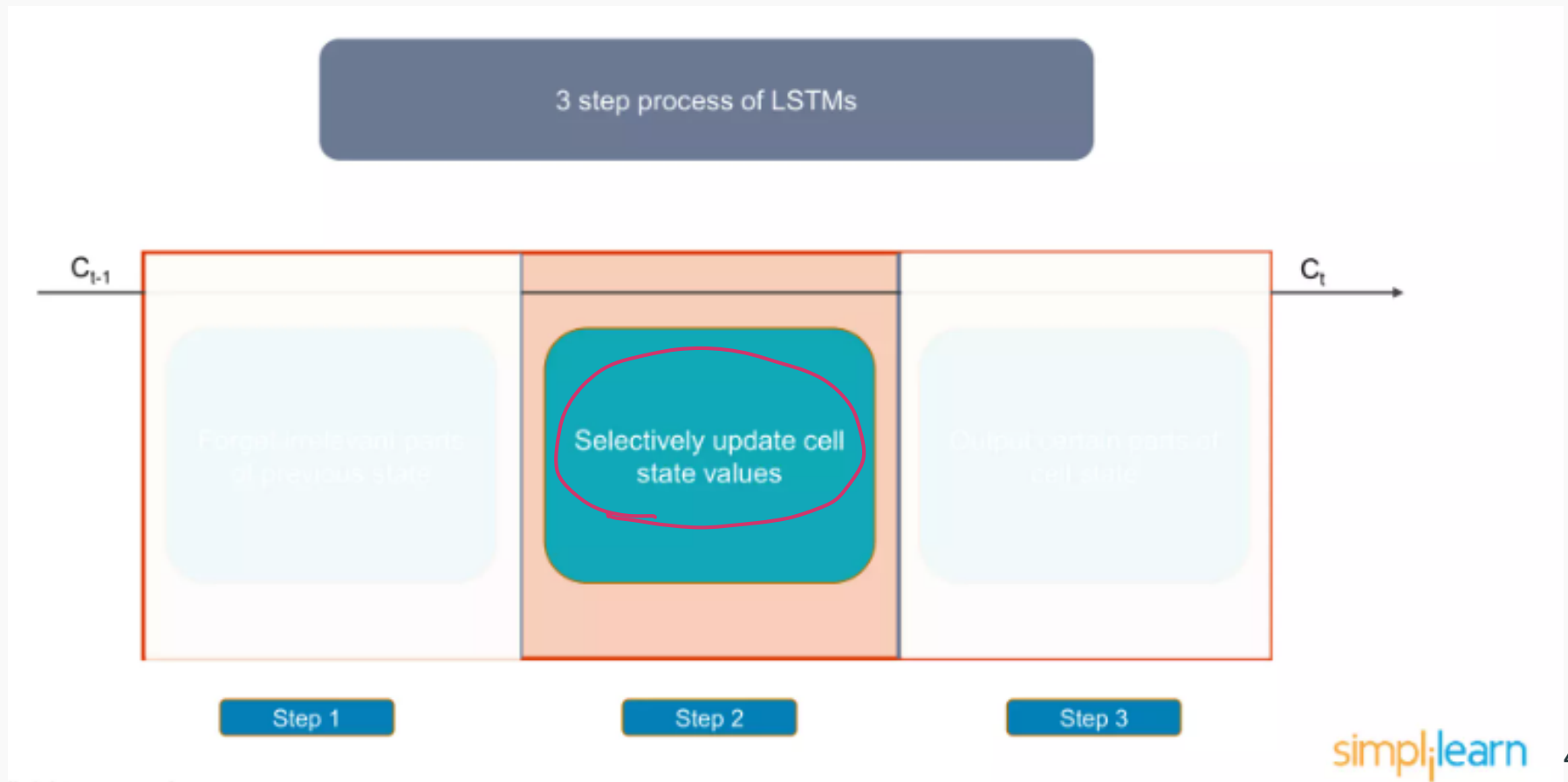
<sup>2</sup><https://www.simplilearn.com/>

# LSTM Architecture (4)



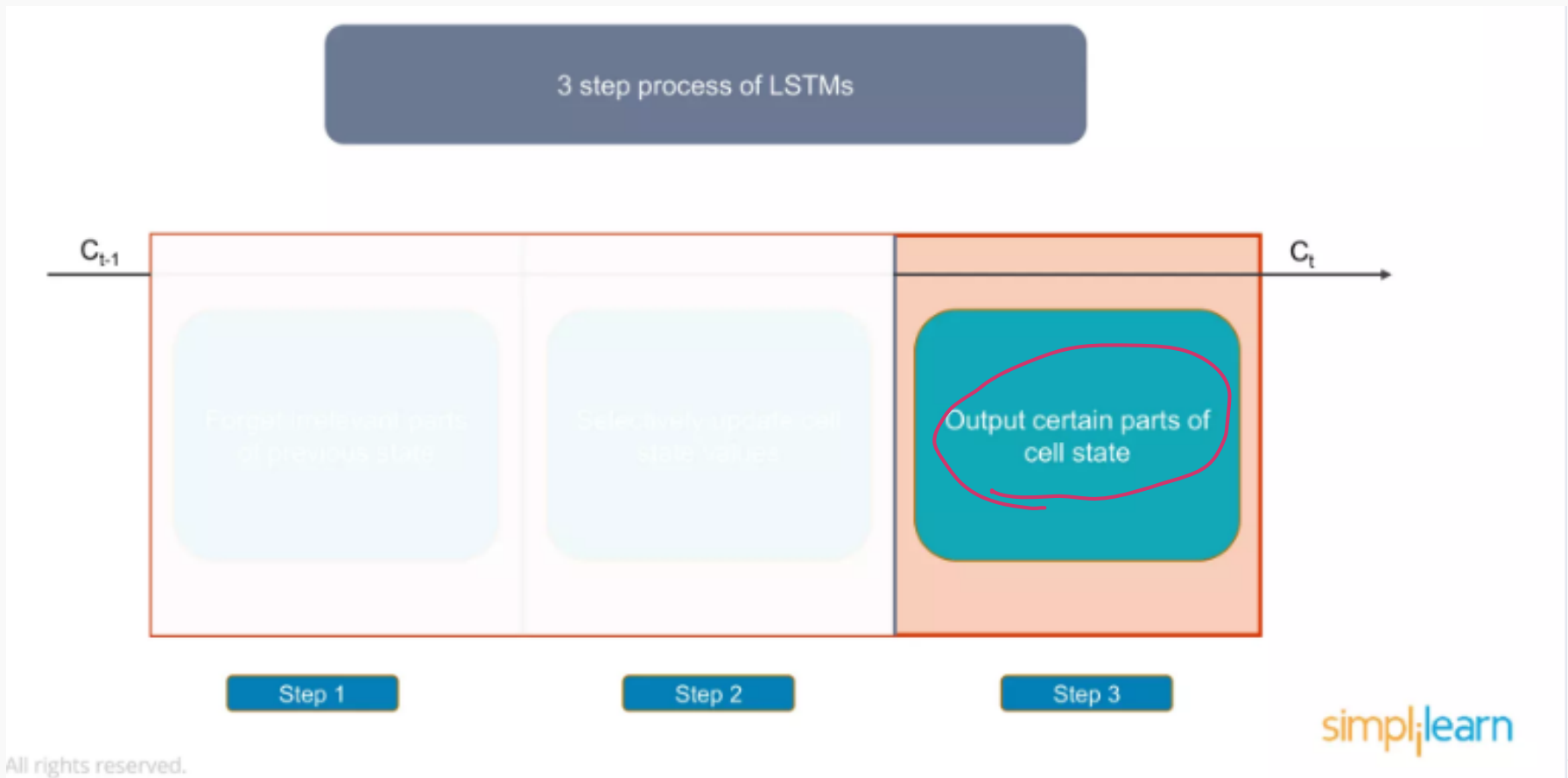
<sup>3</sup><https://www.simplilearn.com/>

# LSTM Architecture (5)



<sup>4</sup><https://www.simplilearn.com/>

# LSTM Architecture (6)



<sup>5</sup> <https://www.simplilearn.com/>

# Working of LSTMs Cell (1)

Step-1

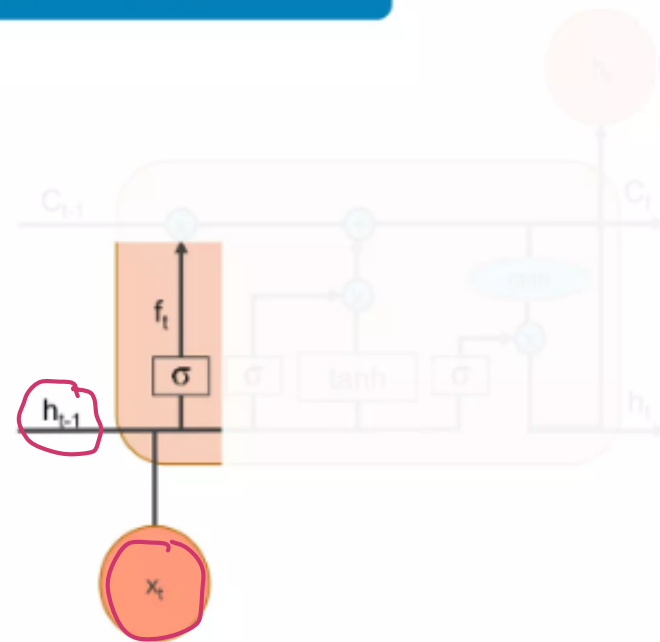
Decides how much of the past it should remember

First step in the LSTM is to decide which information to be omitted in from the cell in that particular time step. It is decided by the sigmoid function. It looks at the previous state ( $h_{t-1}$ ) and the current input  $x_t$  and computes the function.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

$f_t$  = forget gate

Decides which information to delete that is not important from previous time step



simplilearn

6

# Working of LSTMs Cell (2)

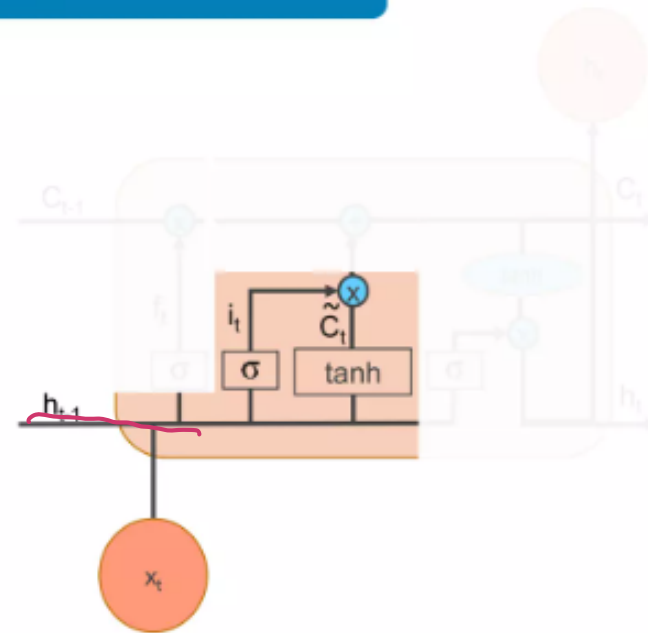
Step-2

Decides how much should this unit add to the current state

In the second layer, there are 2 parts. One is the sigmoid function and the other is the tanh. In the **sigmoid** function, it decides which values to let through (0 or 1). **tanh** function gives the weightage to the values which are passed deciding their level of importance (-1 to 1).

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

$i_t$  = input gate  
Determines which information to let through based on its significance in the current time step



simplilearn

7

<sup>7</sup><https://www.simplilearn.com/>

# Working of LSTMs Cell (3)

Step-3

Decides what part of the current cell state makes it to the output

The third step is to decide what will be our output. First, we run a sigmoid layer which decides what parts of the cell state make it to the output. Then, we put the cell state through tanh to push the values to be between -1 and 1 and multiply it by the output of the sigmoid gate.

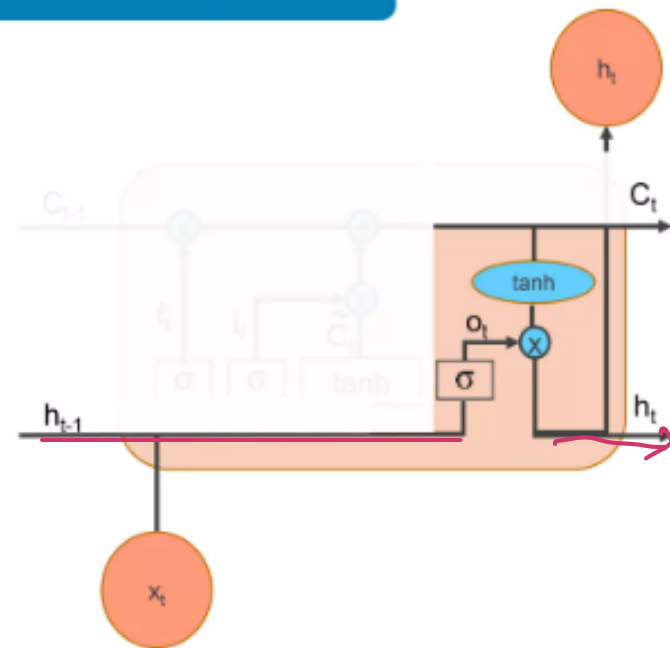
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

$o_t$  = output gate

Allows the passed in information to impact the output in the current time step

$$h_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$



simplilearn 8

<sup>8</sup> <https://www.simplilearn.com/>

# Working of LSTMs Cell (4): LSTM Cell Equations

At time step  $t$ , given input  $x_t$ , previous hidden state  $h_{t-1}$ , and previous cell state  $C_{t-1}$ :

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f) && \text{(Forget Gate)} \\ i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i) && \text{(Input Gate)} \\ \tilde{C}_t &= \tanh(W_C[h_{t-1}, x_t] + b_C) && \text{(Candidate Cell State)} \\ C_t &= f_t \odot C_{t-1} + i_t \odot \tilde{C}_t && \text{(Updated Cell State)} \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o) && \text{(Output Gate)} \\ h_t &= o_t \odot \tanh(C_t) && \text{(Hidden State Output)} \end{aligned}$$

$\sigma$  is the sigmoid activation;  $\odot$  is element-wise multiplication.

$\angle (w)$

# How LSTM Is Trained

- **Forward Pass:**

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

# How LSTM Is Trained

- **Forward Pass:**

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

- **Backward Pass:**

- Gradients of the loss are computed using *Backpropagation Through Time (BPTT)*.
- Gradients flow backward across all time steps and LSTM parameters.
- Optimizer (e.g., Adam) updates weights to minimize the loss.

$$\hat{w}_f, \hat{w}_i, \hat{w}_o$$

# How LSTM Is Trained

- **Forward Pass:**

- Input sequences are passed through the LSTM layer(s).
- At each time step, hidden and cell states are updated using gates.
- The final output is passed to a dense layer to compute predictions.
- A loss (e.g., MSE or cross-entropy) is calculated from predictions.

- **Backward Pass:**

- Gradients of the loss are computed using *Backpropagation Through Time (BPTT)*.
- Gradients flow backward across all time steps and LSTM parameters.
- Optimizer (e.g., Adam) updates weights to minimize the loss.

- 
- **Repeat for multiple epochs** until validation loss stabilizes or early stopping is triggered.
- 

# Advantages of LSTM

- **Handles long sequences:** Able to learn long-term dependencies
- **Gradient flow:** Mitigates vanishing gradient problem
- **Memory mechanism:** Gating allows selective memory and forgetting
- **Versatile:** Works well with sequences of varying length

Compared to traditional RNNs, **LSTMs** are much more stable and effective during training.

# Applications of LSTM

LSTM networks are widely used in sequential data problems:

- Natural Language Processing (NLP):

- Text generation ✓
- Machine translation ✓
- Named entity recognition

- Speech Recognition ✓

- Time Series Forecasting ✓

- Video Analysis ✓

LSTM remains a foundational component in many deep learning applications involving temporal data.

## 3. LSTM: Long Short-Term Memory

### 3.1 Motivation

### 3.2 LSTM Architecture

### 3.3 Implementing LSTMs in Tensorflow

# Implementing LSTMs in TensorFlow

We demonstrate how to build a simple LSTM using `tf.keras` to perform time series prediction.

Open the Google Colab notebook from the Link:

[https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module4/code/NNs\\_introduction\\_to\\_lstm.ipynb](https://github.com/yadavrishikesh/Masterclass-IITMandi/blob/main/Module4/code/NNs_introduction_to_lstm.ipynb)

# Preparing Time Series Data for LSTM

- LSTM expects input as 3-D tensors: (samples, timesteps, features).
- Convert raw time series into sequences using a sliding window.

```
1 import numpy as np
2
3 def create_sequences(data, window_size):
4     X, y = [], []
5     for i in range(len(data) - window_size):
6         X.append(data[i:i+window_size])
7         y.append(data[i+window_size])
8     return np.array(X), np.array(y)
9
10 X, y = create_sequences(my_series, window_size=10)
11 X = X[..., np.newaxis]          # Add feature dim for
    univariate
```

# LSTM Model Architecture

```
1 from tensorflow.keras.models import Sequential
2 from tensorflow.keras.layers import LSTM, Dense
3
4 model = Sequential([
5     LSTM(64, activation='tanh',
6         input_shape=(X.shape[1], X.shape[2])),
7     Dense(1) # Predict next value
8 ])
9
10 model.compile(optimizer='adam', loss='mse')
11 model.summary()
```

- Stack additional LSTMs or add dropout if needed.
- Adjust the output layer for regression or classification tasks.

# Model Training and Evaluation

```
1 history = model.fit(  
2     X_train, y_train,  
3     validation_data=(X_val, y_val),  
4     epochs=20,  
5     batch_size=32,  
6     verbose=1  
7 )  
8  
9 loss = model.evaluate(X_test, y_test)  
10 print(f"Test loss: {loss:.3f}")
```

- Monitor validation loss to avoid over-fitting.
- Consider callbacks like EarlyStopping or ReduceLROnPlateau.

# Making Forecasts

```
1 last_sequence = X_test[-1].reshape(1, X.shape[1], X.shape  
   [2])  
2 predicted = model.predict(last_sequence)  
3 print(f"Predicted next value: {predicted[0,0]:.2f}")
```

- Roll forward one step at a time for multi-step forecasts.
- Feed predictions back as input for iterative forecasting.